RV College of Engineering®

UNIT-III
Main Memory Management



RV College of Engineering®

Chapter 8: Memory Management

- Background
- Swapping
- Contiguous Memory Allocation
- Segmentation
- Paging
- Structure of the Page Table

RV College of Engineering®

Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques, including paging and segmentation

RV College of Engineering®

Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers built into processor are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Background

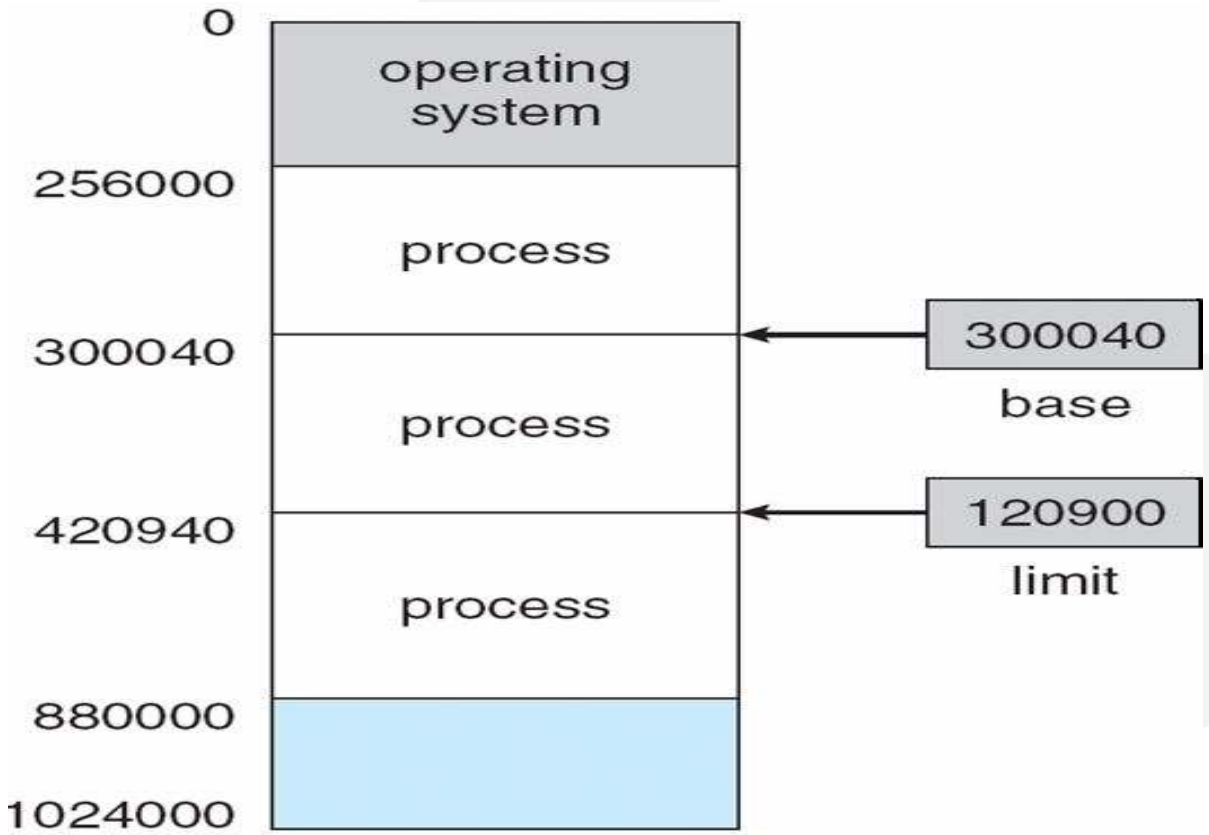
- Along with speed of access , protection of OS by user process and one process accessing another process is needed .
- This form of protection is provided at hardware level.
- first requirement is to make sure that each process has a separate memory space
- Separate per-process memory space protects the processes from each other

Background

- To separate memory spaces, we need the ability to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.
- this protection can be provided by using two registers, usually a base and a limit,
- The base register holds the smallest legal physical memory address; the limit register specifies the size of the range

Base and Limit Registers

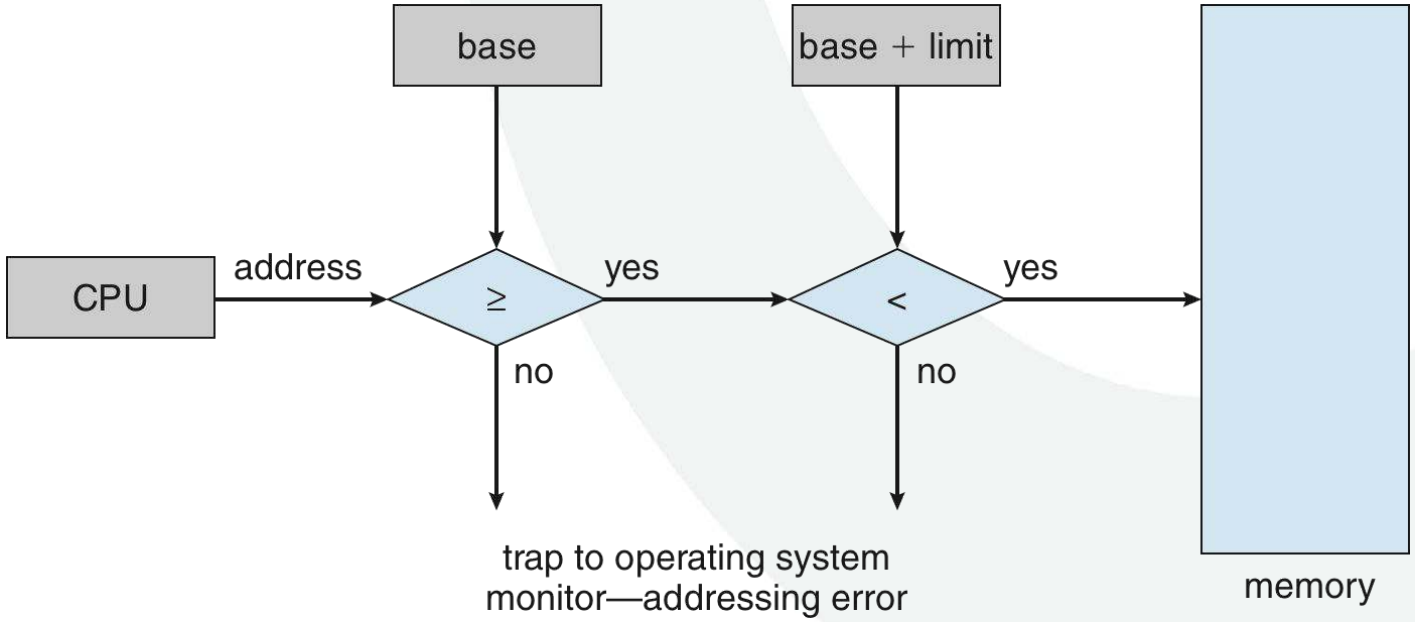
- A pair of **base** and **limit** registers define the logical address space



Protection

- Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers.
- Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system,
- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.

Hardware Address Protection with Base and Limit Registers



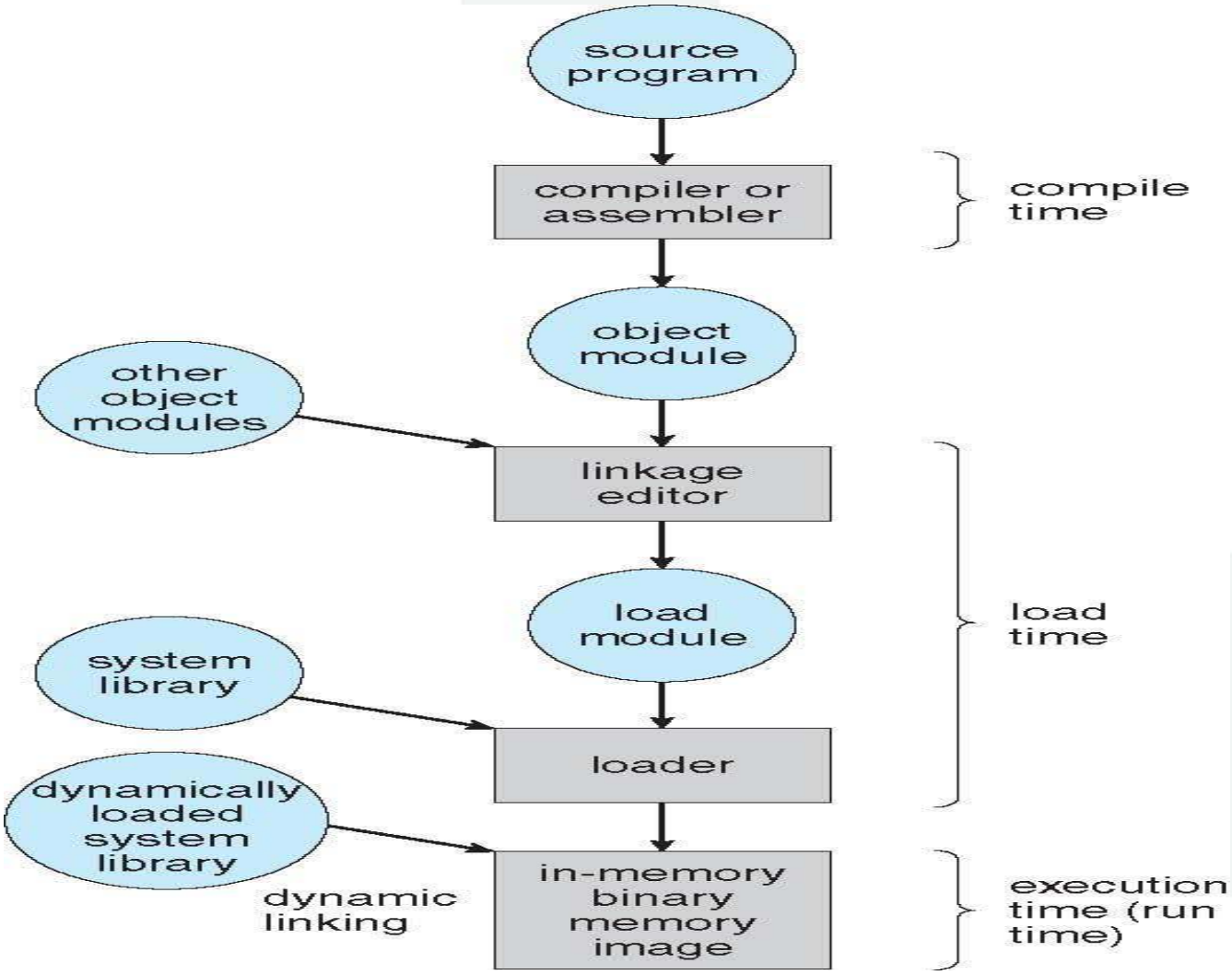
RV College of Engineering®

Address Binding

- program resides on a disk as a binary executable file.
- Depending on the memory management in use, the process may be moved between disk and memory during its execution.
- the processes on the disk forms the input queue

RV College of Engineering®

Multistep Processing of a User Program



Address Binding

- Inconvenient to have first user process physical address always at 0000
- Further, addresses represented in different ways at different stages of a program's life
  - Source code addresses usually symbolic-example variables
  - Compiled code addresses **bind** to relocatable addresses
    - i.e. "14 bytes from beginning of this module"
  - Linker or loader will bind relocatable addresses to absolute addresses
    - i.e. 74014
  - Each binding maps one address space to another

Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
  - **Compile time**: If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
  - **Load time**: Must generate **relocatable code** if memory location is not known at compile time
  - **Execution time**: Binding delayed until run time if the process can be moved during its execution from one memory segment to another
    - Need hardware support for address maps (e.g., base and limit registers)

RV College of Engineering®

Logical Versus Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit, loaded into the **Memory Address Register**

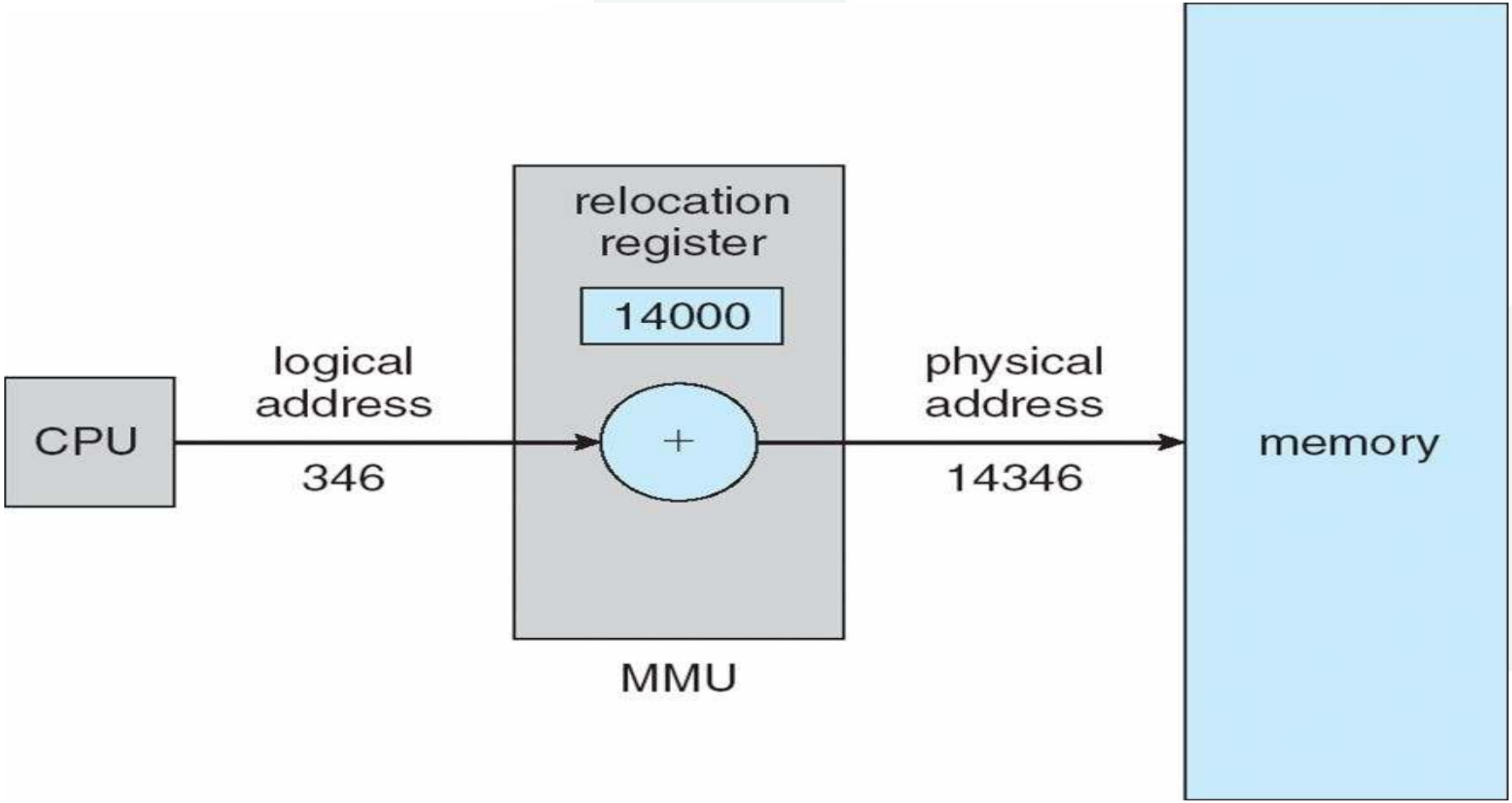
Logical Versus Physical Address Space

- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated corresponding to logical addresses

Logical Versus Physical Address Space

- The run-time mapping from virtual to physical addresses is done by a hardware device called the **memory-management unit (MMU).**

Dynamic relocation using a relocation register.



Dynamic relocation using a relocation register.

- Hardware device that at run time maps virtual to physical address
- Many methods possible
- Simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
  - Base register now called **relocation register**
  - MS-DOS on Intel 80x86 used 4 relocation registers

Dynamic relocation using a relocation register.

- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses
  - The final location of a referenced memory address is not determined until the reference is made.
  - We have two different types of addresses: logical addresses (in the range 0 to max) and physical addresses (in the range R + 0 to R + max for a base value R).

Dynamic relocation using a relocation register.

- the concept of a logical address space that is bound to a separate physical address space is central to proper memory management.

Dynamic Loading

- With dynamic loading, a routine is not loaded until it is called.
- All routines are kept on disk in a relocatable load format.
- Better memory-space utilization; unused routine is never loaded
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading

Dynamic Linking and Shared Libraries

- Dynamically linked libraries are system libraries that are linked to user programs when the programs are run.
- Some operating systems support only static linking, in which system libraries are treated like any other object module and are combined by the loader into the binary program image.
- Dynamic linking, in contrast, is similar to dynamic loading
- Ex: language subroutine libraries

Dynamic Linking and Shared Libraries

- With dynamic linking, a stub is included in the image for each library- routine reference.
- Small piece of code, *stub*, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Unlike dynamic loading, dynamic linking and shared libraries generally require help from the operating system

RV College of Engineering®

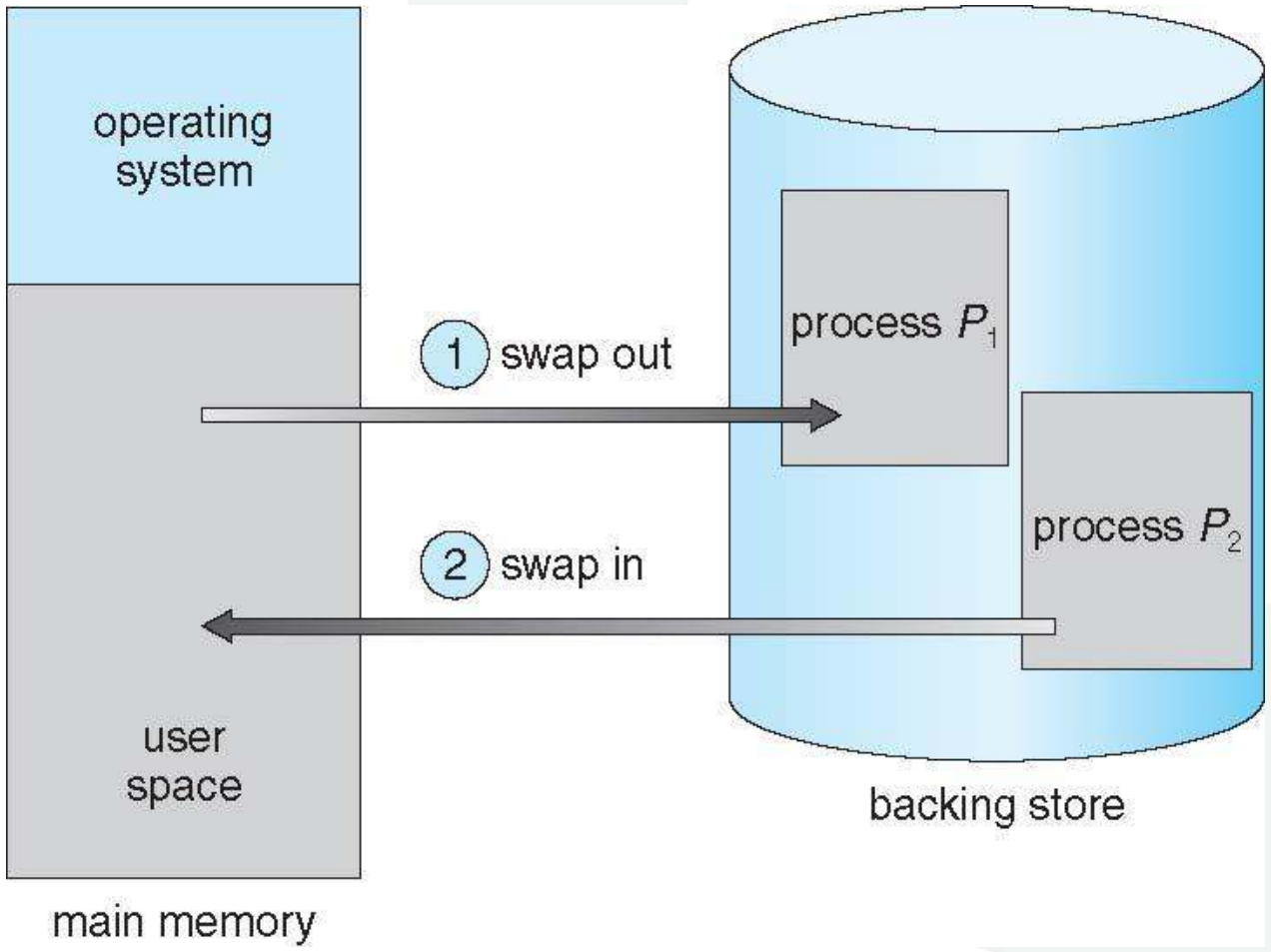
Comparison

Static Linking	Dynamic Linking
necessary library functions are embedded directly in the program's executable binary file.	The process of linking external libraries and references at runtime, when the program is loaded or executed.
Static linking is performed during the compilation of source program	Run time linking
Every program generated must contain copies of exactly the same common system library functions	Memory requirements of the program are reduced
Requires recompilation	Changes can be made without recompilation

Swapping

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- Backing store – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

Schematic View of Swapping



Standard Swapping

- Whenever the CPU scheduler decides to execute a process, it calls the dispatcher. The dispatcher checks to see whether the next process in the queue is in memory. If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process.

Swapping

- Swapping time is calculated as:
- the user process is 100 MB in size and the backing store is a standard hard disk with a transfer rate of 50 MB per second. The actual transfer of the 100-MB process to or from main memory takes
- 100 MB/50 MB per second = 200mseconds
- total swap time is about 4,000 milliseconds

Swapping

- the major part of the swap time is transfer time.
- The total transfer time is directly proportional to the amount of memory swapped.
- system with 4 GB of main memory and a resident operating system taking 1 GB, the maximum size of the user process is 3 GB.
- A 100-MB process could be swapped out in 2 seconds, compared with the 60 seconds required for swapping 3 GB.
- Swapping is to be done ,we must be sure that it is completely idle.

Swapping

- There are two main solutions to this problem: never swap a process with pending I/O , or execute I/O operations only into operating-system buffers.
- Standard swapping is not used in modern operating systems.
- One modified option is : swapping is normally disabled but will start if the amount of free Memory falls below a threshold amount.
- Another variation involves swapping portions of processes—rather than entire processes—to decrease swap time.

Swapping on Mobile Systems

- mobile systems typically do not support swapping in any form.
- Mobile devices generally use flash memory.
- the resulting space constraint avoids swapping .
- limited number of writes that flash memory can tolerate before it becomes unreliable and the poor throughput between main memory and flash memory in these devices.

Swapping

- Apple’s iOS asks applications to voluntarily relinquish allocated memory.
- Android may terminate a process if insufficient free memory is available.

Contiguous Memory Allocation

- One of the efficient method for allocating main memory among OS and user processes.
- Memory Protection: achieved through relocation register and limit register
- Relocation register also provides support for transient operating system code feature .

RV College of Engineering®

Memory Allocation

- Simplest methods for allocating memory is to divide memory into several fixed-sized partitions.
- The degree of multiprogramming is bound by the number of partitions. Originally used by the IBM OS/360 operating system (called MFT)
- Multiprogramming with a Fixed number of Tasks
- MVT a variant of MFT is used primarily in batch environments.
- Ideas of MVT also applicable to time sharing systems

RV College of Engineering®

Memory Allocation

- In the variable-partition scheme, the operating system keeps a table indicating which parts of memory are available and which are occupied.Initially called **hole.**

Memory Allocation

- As processes enter the system, they are put into an input queue.
- The operating system takes into account the memory requirements of each process
- Calculates the amount of available memory space in determining which processes are allocated memory.
- When a process is allocated space, it is loaded into memory, and it can then compete for CPU time

Memory Allocation

- At any given time, then, we have a list of available block sizes and an input queue.
- The operating system can order the input queue according to a scheduling algorithm.
- Memory is allocated to processes until, finally, the memory requirements of the next process cannot be satisfied.
- Dynamic storage- allocation problem, which concerns how to satisfy a request of size n from a list of free holes.

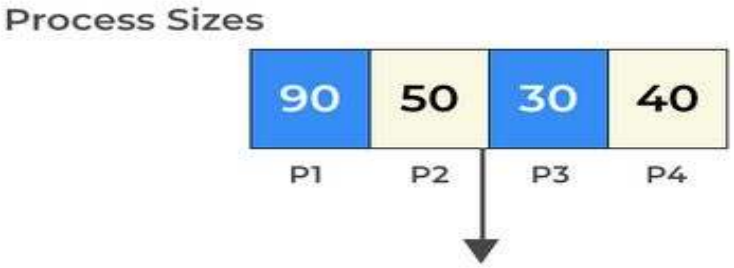
Memory Allocation

- **first-fit, best-fit, and worst-fit** strategies are the ones most commonly used to select a free hole from the set of available holes.
- **First fit.** Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.

Memory Allocation

- **Best fit.** Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
- **Worst fit**. Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

First Fit Allocation in OS



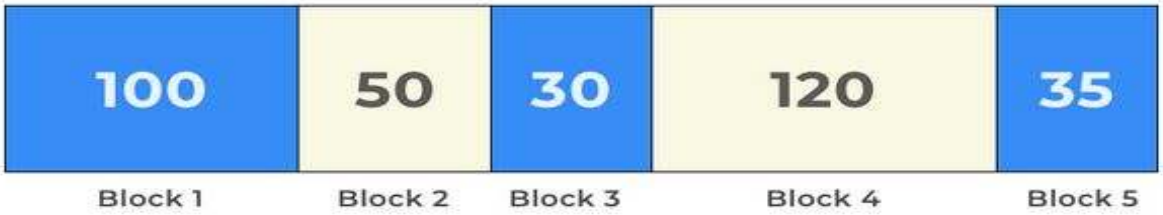
	Size	Allocated to	Memory Wastage After Process Occupies	
Process 1	90	Block2	100 - 90 = 10	
Process 2	50	Block 4	200 - 50 = 150	P4 remains Unallocated
Process 3	30	Block 3	40 - 30 - 10	
Process 4	40	Unallocated	-	

Best Fit Allocation in OS



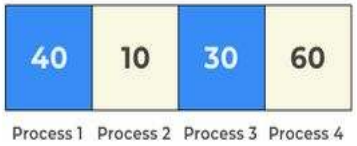


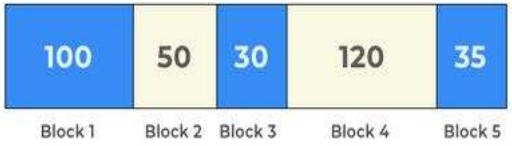
Best FIT Allocation



	Size	Can Occupy?	Memory Wastage After Process Occupies	
Block 1	100	Yes	100 - 40 = 60	
Block 2	50	Yes	50 - 40 = 10	Best
Block 3	30	No	-	
Block 4	120	Yes	120 - 40 = 80	
Block 5	35	No	-	

Best Fit Allocation in OS



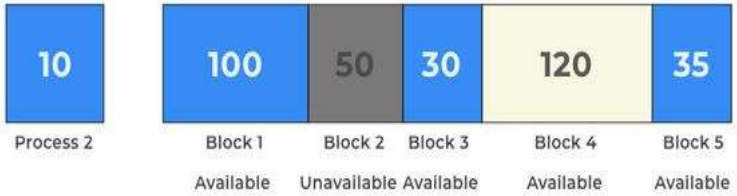




Block / Size	Available	Can Occupy P1?	Memory Wastage After Process Occupies	
Block 1 (100K)	Yes	Yes	100 - 40 = 60	
Block 2 (50K)	Yes	Yes	50 - 40 = 10	Best
Block 3 (30K)	Yes	No	-	
Block 4 (120K)	Yes	Yes	120 - 40 = 80	
Block 5 (35K)	Yes	No	-	

Block 2 results in the least memory wastage

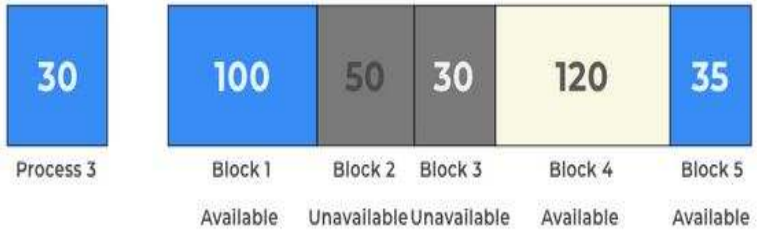
P1 goes to Block 2



Block / Size	Available	Can Occupy P2?	Memory Wastage After Process Occupies	
Block 1 (100K)	Yes	Yes	100 - 10 = 90	
Block 2 (50K)	No	-	-	
Block 3 (30K)	Yes	Yes	30 - 10 = 20	Best
Block 4 (120K)	Yes	Yes	120 - 10 = 110	
Block 5 (35K)	Yes	Yes	35 - 10 = 25	

Block 3 results in the least memory wastage

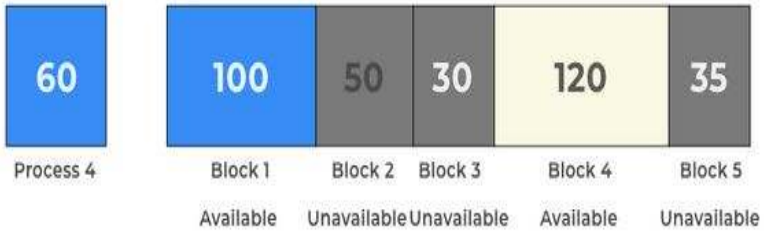
P2 goes to Block 3



Block / Size	Available	Can Occupy P3?	Memory Wastage After Process Occupies	
Block 1 (100K)	Yes	Yes	100 - 30 = 70	
Block 2 (50K)	No	-	-	
Block 3 (30K)	Yes	-	-	
Block 4 (120K)	Yes	Yes	120 - 30 = 90	
Block 5 (35K)	Yes	Yes	35 - 30 = 5	Best

Block 5 results in the least memory wastage

P3 goes to Block 5



Block / Size	Available	Can Occupy P3?	Memory Wastage After Process Occupies	
Block 1 (100K)	Yes	Yes	100 - 60 = 40	Best
Block 2 (50K)	No	-	-	
Block 3 (30K)	Yes	-	-	
Block 4 (120K)	Yes	Yes	120 - 60 = 60	
Block 5 (35K)	Yes	-	-	

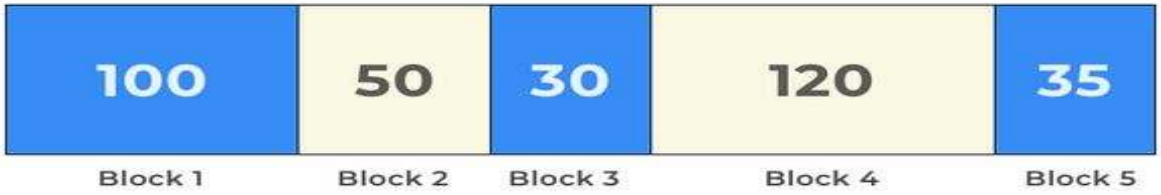
Block 1 results in the least memory wastage

P4 goes to Block 1

Worst Fit Allocation in OS



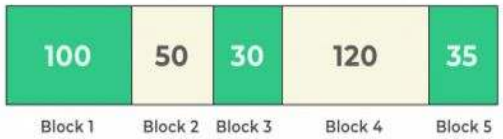
Worst FIT Allocation



	Size	Can Occupy?	Memory Wastage After Process Occupies	
Block 1	100	Yes	100 - 40 = 60	
Block 2	50	Yes	50 - 40 = 10	
Block 3	30	No	-	
Block 4	120	Yes	120 - 40 = 80	Worst
Block 5	35	No	-	

Worst Fit Allocation in OS Example

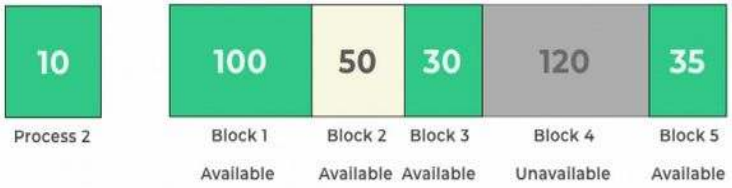






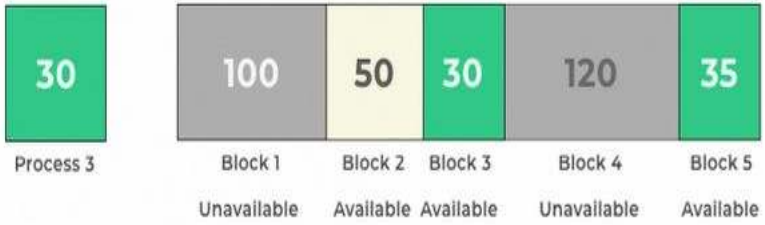
	Available	Can Occupy P1?	Memory Wastage After Process Occupies	
Block 1	Yes	Yes	100 - 40 = 60	
Block 2	Yes	Yes	50 - 40 = 10	
Block 3	Yes	No	-	
Block 4	Yes	Yes	120 - 40 = 80	Worst
Block 5	Yes	No	-	

P1 goes to Block 4



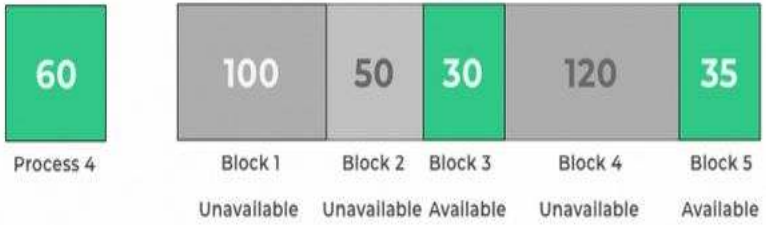
	Available	Can Occupy P2?	Memory Wastage After Process Occupies	
Block 1	Yes	Yes	100 - 10 = 90	Worst
Block 2	Yes	Yes	50 - 10 = 40	
Block 3	Yes	Yes	30 - 10 = 20	
Block 4	No			
Block 5	Yes	Yes	35 - 10 = 25	

P2 goes to Block 1



	Available	Can Occupy P3?	Memory Wastage After Process Occupies	
Block 1	No			
Block 2	Yes	Yes	50 - 30 = 20	Worst
Block 3	Yes	Yes	30 - 30 = 0	
Block 4	No			
Block 5	Yes	Yes	35 - 30 = 5	

P3 goes to Block 2



	Available	Can Occupy P4?	Memory Wastage After Process Occupies
Block 1	No		
Block 2	No		
Block 3	Yes	No	
Block 4	No		
Block 5	Yes	No	

P4 remains unallocated, As can not be fit by any Free blocks

Example

- blockSize[] = {100, 500, 200, 300, 600}; processSize[] = {212, 417, 112, 426};

RV College of Engineering®

Implementation

- 1- Input memory blocks and processes with sizes. 2- Initialize all memory blocks as free.
- 3- Start by picking each process and find the minimum block size that can be assigned to current process i.e., find min(bockSize[1], blockSize[2],.....blockSize[n]) > processSize[current], if found then assign it to the current process.
- 4- If not then leave that process and keep checking the further processes.

Fragmentation

- Fragmentation refers to **problem that occurs in the OS in which a process is unloaded and loaded from memory, and the free memory space gets fragmented**
- As processes are loaded and removed from memory, the free memory space is broken into little pieces.
- External fragmentation exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous:

RV College of Engineering®

Fragmentation

- storage is fragmented into a large number of small holes.
- Depending on the total amount of memory storage and the average process size, external fragmentation may be a minor or a major problem.
- given N allocated blocks, another 0.5 N blocks will be lost to fragmentation. That is, one-third of memory may be unusable! This property is known as the 50-percent rule.
- The 50 Percent Rule in memory fragmentation states that for N allocated memory blocks, roughly 0.5N more blocks (holes) are lost to external fragmentation, meaning about one-third (N + 0.5N = 1.5N total, so 0.5N/1.5N = 1/3) of memory becomes unusable as tiny, non-contiguous spaces

Example

- **Total Memory**: 100 MB
- **Allocated Memory**: 50 MB (50% rule)
- **System Overhead**: 10 MB (for OS and background services)
- **Fragmentation**: 7 MB (not usable due to fragmentation)

Example

- In this scenario:
- Total memory allocated for application: 50 MB
- Total memory taken by system overhead: 10 MB
- Usable memory after considering fragmentation: 50 MB - 10 MB - 7 MB = 33 MB
- Thus, the application effectively has access to only about 33% of the total memory, illustrating how the 50 percent rule can lead to a scenario where only a third of memory is available for actual use.

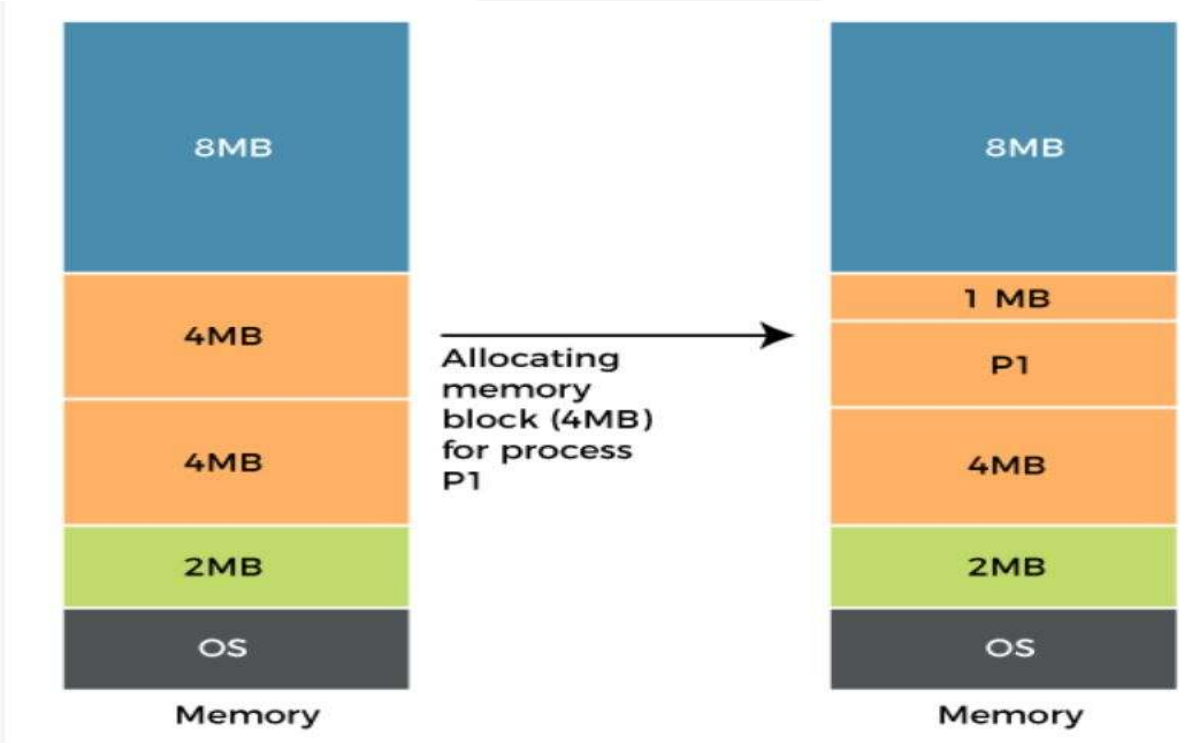
Fragmentation

- Fragmentation can be:
- Internal:Internal fragmentation occurs when the memory is distributed into fixed-sized blocks. If the memory allocated to the process is slightly larger than the memory demanded, then the difference between allocated and demanded memory is known as internal fragmentation.
- External :External fragmentation is a phenomenon that occurs when free memory is split into small blocks and interspersed with allocated memory

RV College of Engineering®

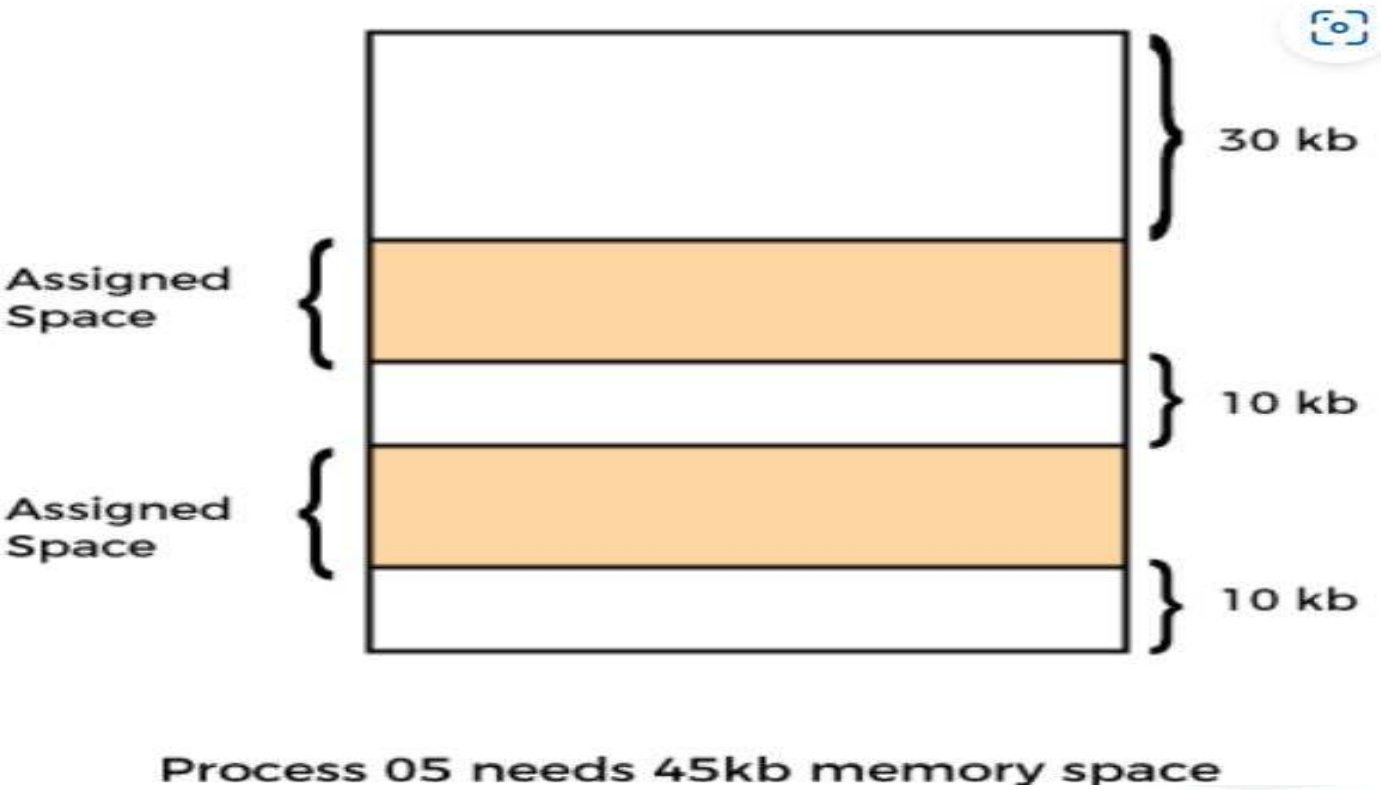
Internal Fragmentation

- Assume that memory allocation in RAM is done using fixed partitioning:**2MB, 4MB, 4MB**, and **8MB** are the available sizes.



RV College of Engineering®

External fragmentation



Process 05 needs 45kb memory space

Internal fragmentation	External fragmentation
Fixed sized memory blocks	Variable sized memory blocks
When process is small	When process is removed
The solution of internal fragmentation is the best-fit block	The solution to external fragmentation is compaction and paging

External fragmentation

- One solution to the problem of external fragmentation is compaction. The goal is to shuffle the memory contents so as to place all free memory together in one large block.
- Compaction algorithms:
- move all processes toward one end of memory;
- permit the logical address space of the processes to be noncontiguous

Compaction solutions:

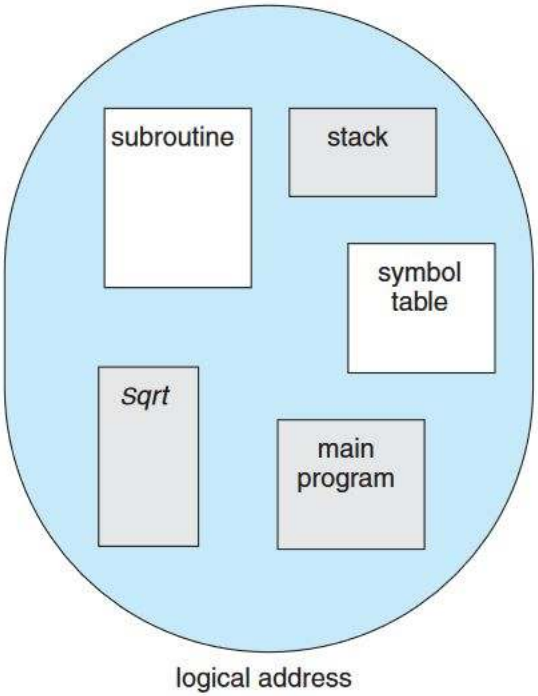
- **Segmentation**
- **Paging**

Segmentation

- Support of hardware to provide a memory mechanism that mapped the programmer's view to the actual physical memory.
- Programmer has a view of memory
- Rather than just array of blocks
- Segmentation is a memory-management scheme that supports this programmer view of memory.
- A logical address space is a collection of segments.
- Each segment has a name and a length.

RV College of Engineering®

Programmer view of memory



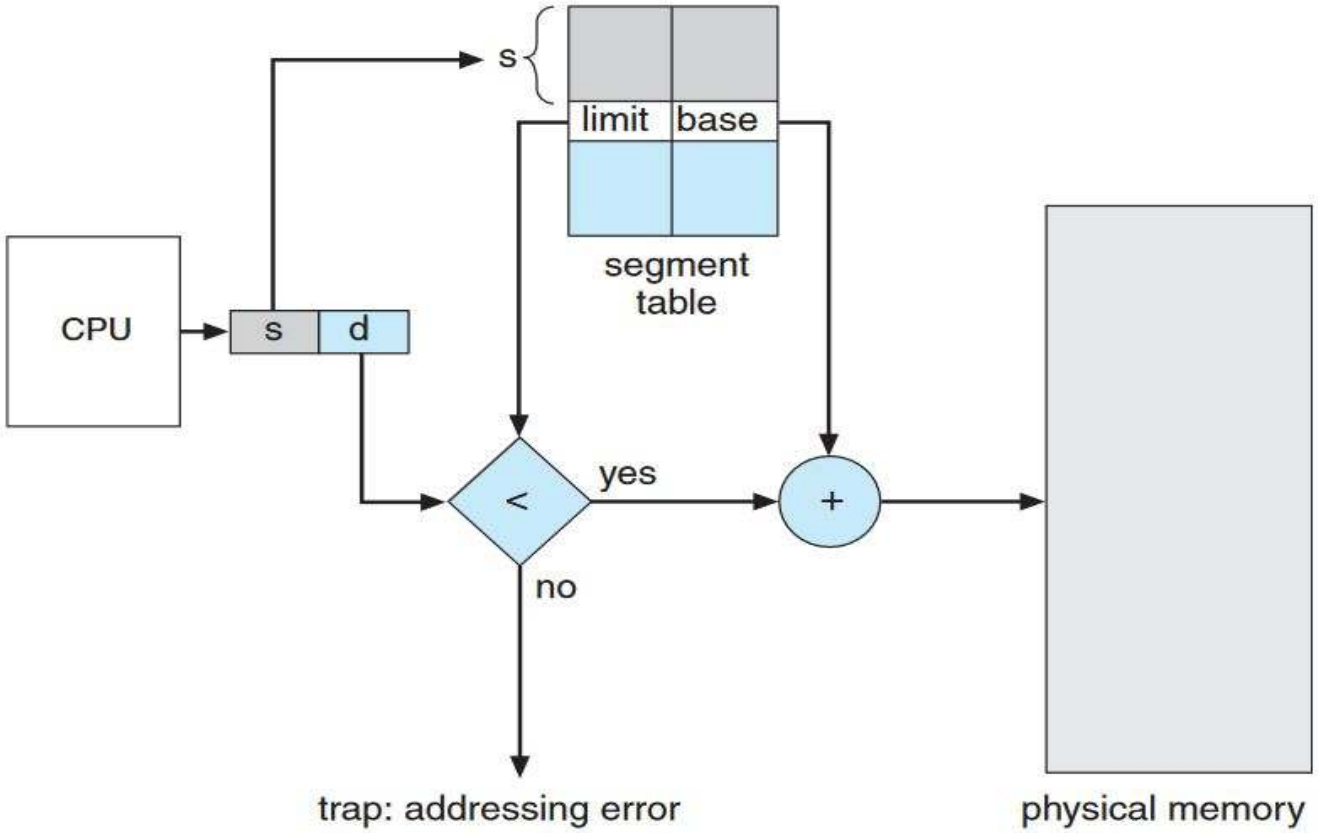
RV College of Engineering®

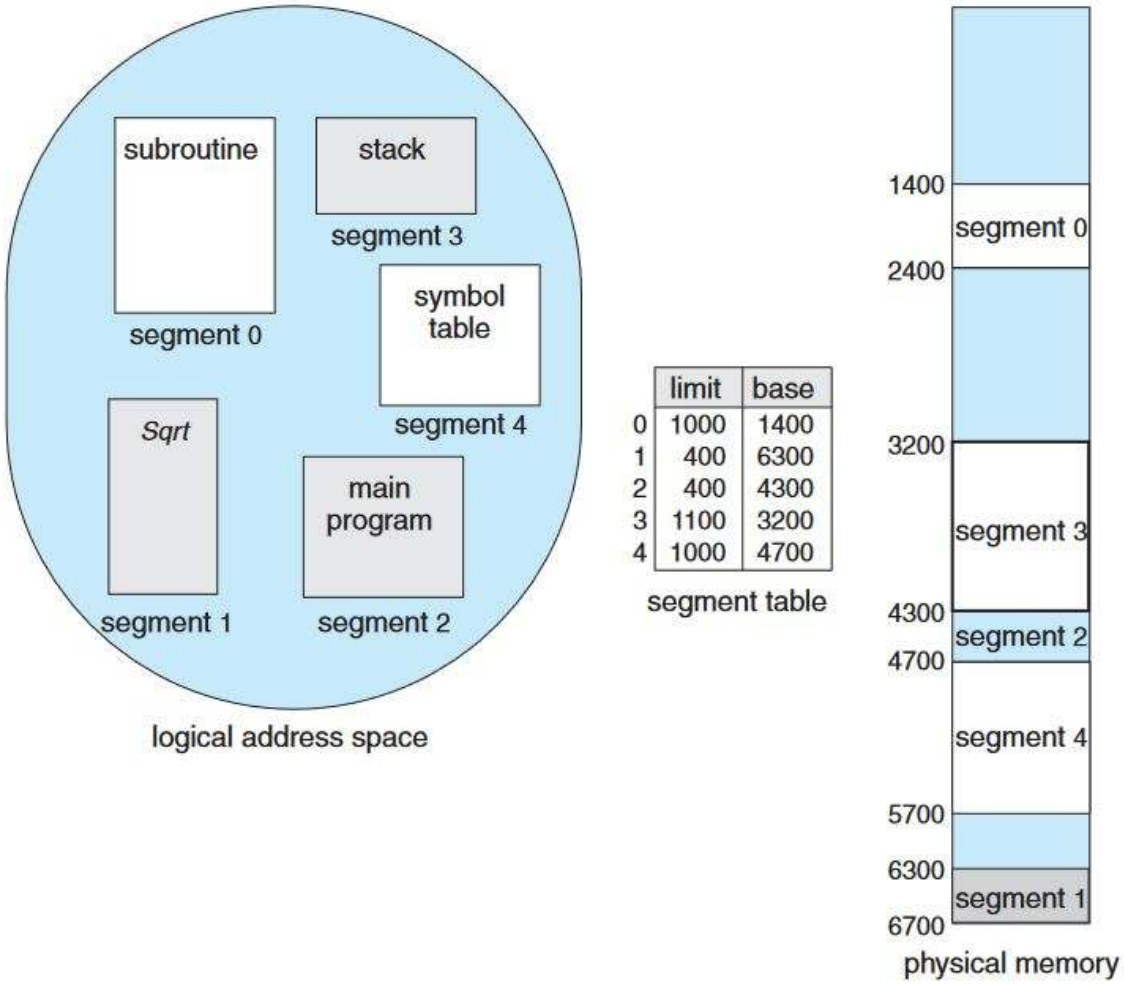
Segmentation

- The addresses specify both the segment name and the offset within the segment.
- The programmer therefore specifies each address by two quantities: a segment number and an offset.
    - <segment-number, offset>.

RV College of Engineering®

Segmentation Hardware




subroutine
stack
segment 3
symbol table
segment 0
segment 4
Sqrt
main program
segment 1
segment 2
logical address space
limit
base
0 1000 1400
1 400 6300
2 400 4300
3 1100 3200
4 1000 4700
segment table
1400
segment 0
2400
3200
segment 3
4300
segment 2
4700
segment 4
5700
6300
segment 1
6700
physical memory

Paging

- Segmentation permits the physical address space of a process to be non-contiguous.
- The basic method for implementing paging involves breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of the same size called pages
- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available

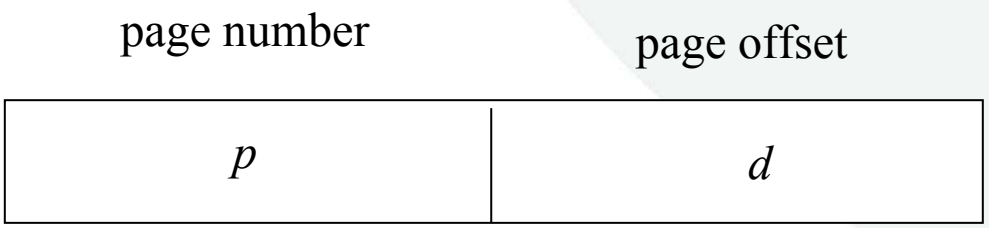
Paging

- Keep track of all free frames
- To run a program of size *N* pages, need to find *N* free frames and load program
- Set up a **page table** to translate logical to physical addresses

RV College of Engineering®

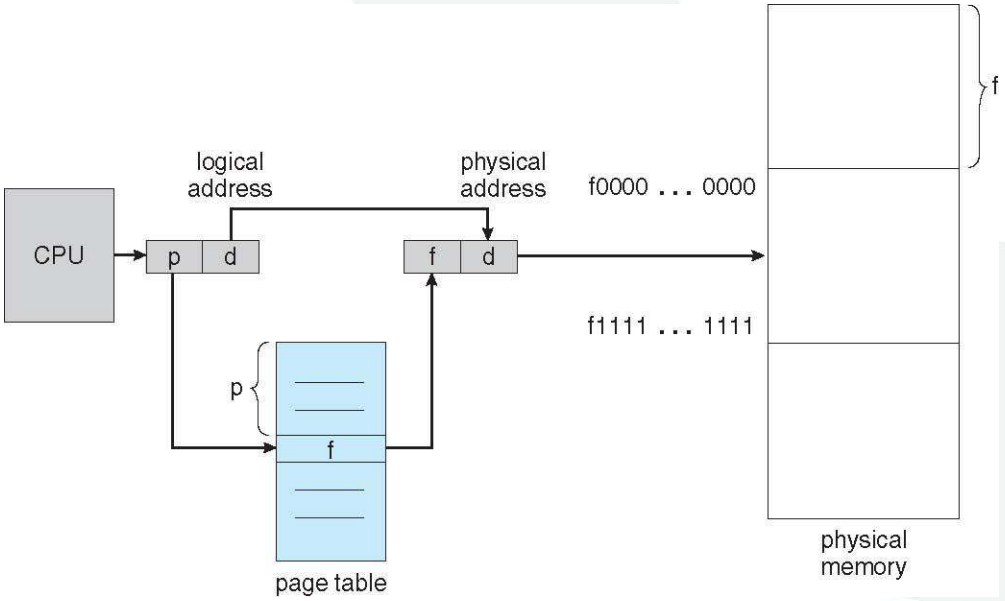
Address Translation Scheme

- Address generated by CPU is divided into:
  - **Page number (*p*)** – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset (*d*)** – combined with base address to define the physical memory address that is sent to the memory unit



RV College of Engineering®

Paging hardware



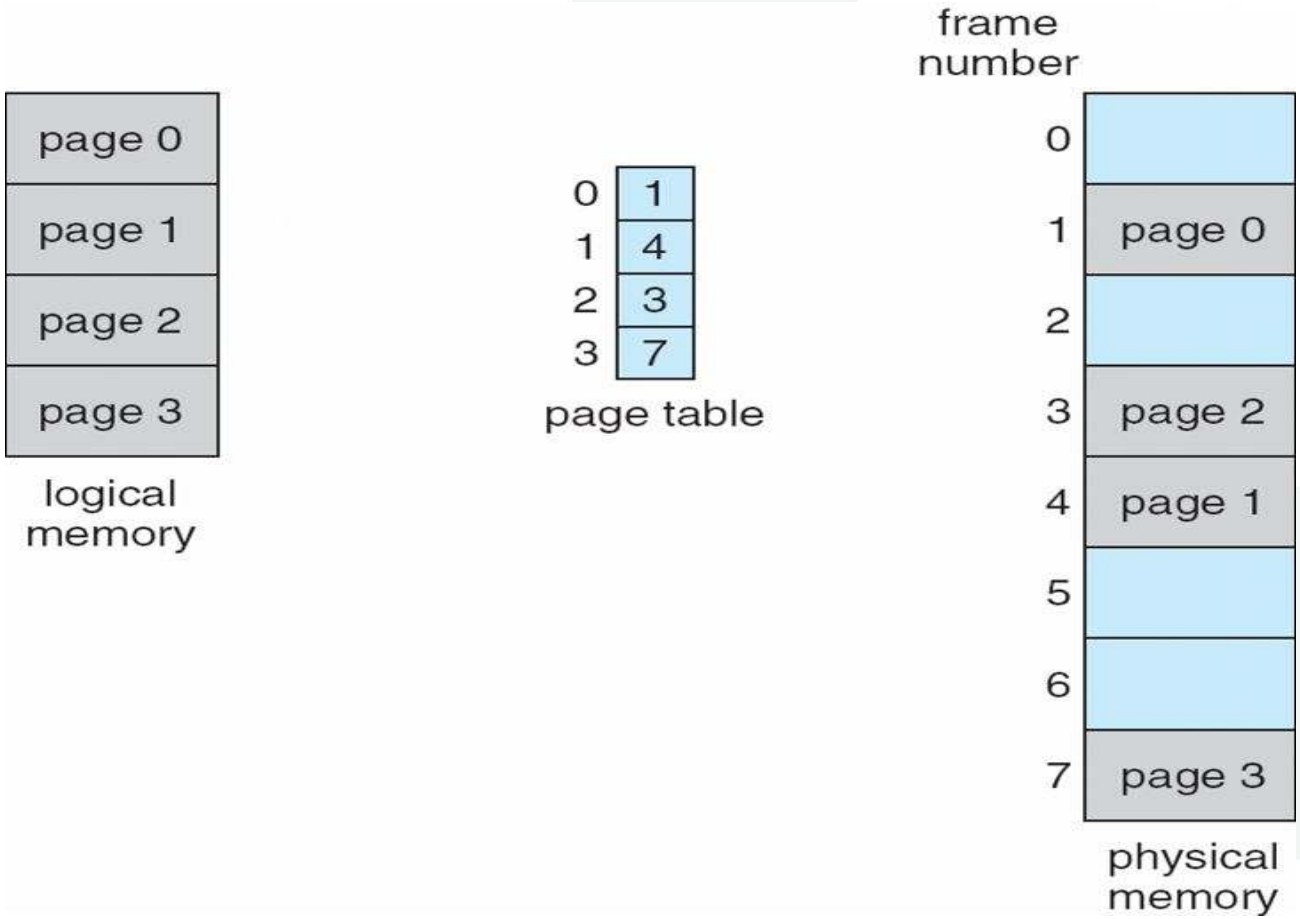
Paging

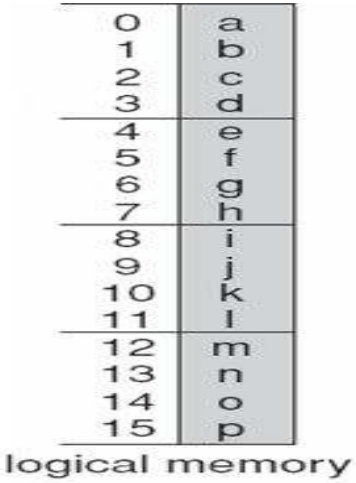
- When a process is to be executed, its pages are loaded into any available memory frames from their source-file system, backing store.
- The backing store is divided into fixed-sized blocks that are the same size as the memory frames or clusters of multiple frames.

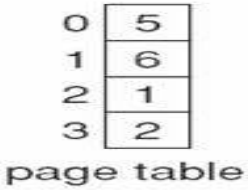
Paging

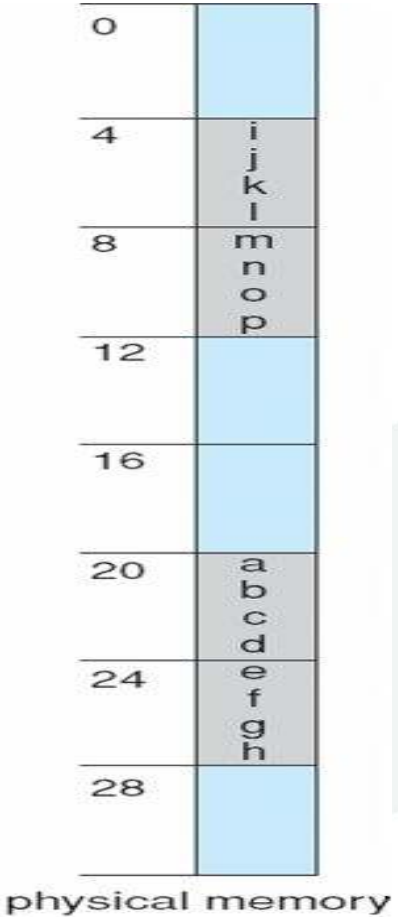
- Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d)
- The page number is used as an index into a page table.
- The page table contains the base address of each page in physical memory.
- This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.
- The page size (like the frame size) is defined by the hardware.

page size of 4 bytes and a physical memory of 32 bytes (8 pages)




0 a
1 b
2 c
3 d
4 e
5 f
6 g
7 h
8 i
9 j
10 k
11 l
12 m
13 n
14 o
15 p
logical memory


0 5
1 6
2 1
3 2
page table


0
4 i j k l
8 m n o p
12
16
20 a b c d
24 e f g h
28
physical memory

RV College of Engineering®

Paging (Cont.)

- Paging suffers internal fragmentation
- Calculating internal fragmentation
  - Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes
  - Internal fragmentation of 2,048 - 1,086 = 962 bytes
  - In the worst case, a process would need n pages plus 1 byte. It would be allocated n + 1 frames, resulting in internal fragmentation of almost an entire frame.
  - pages typically are between 4 KB and 8 KB in size,

Paging vs. Segmentation

Paging

- Invisible from the user program
- Divides physical memory into blocks (pages)
- Pages are of uniform, predetermined size
- Logical address is a positive integer

Segmentation

- Visible to the user program
- Divides logical memory into blocks (segments)
- Segments are of arbitrary sizes
- Logical address is an ordered pair: (segment, offset)

In both cases, the operating system works together with the hardware to translate logical addresses into physical addresses.

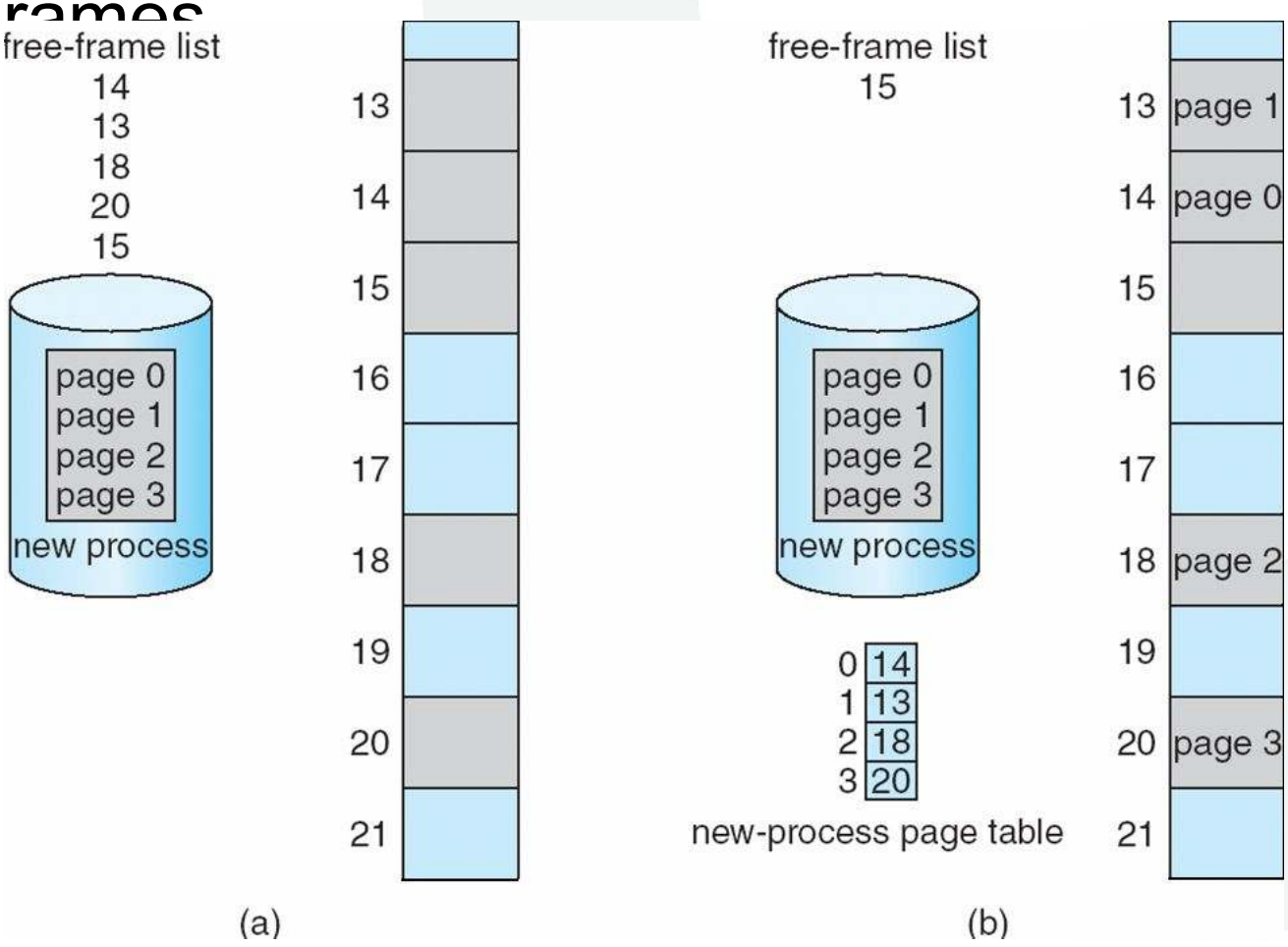
RV College of Engineering®

Paging (Cont.)

- which frames are allocated, which frames are available, how many total frames there are, and so on. This information is generally kept in a data structure called a frame table

RV College of Engineering®

Free Frames



Before allocation

After allocation

RV College of Engineering®

Hardware Support

- Each operating system has its own methods for storing page tables.
- allocate a page table for each process, A pointer to the page table is stored with the other register values.
- The hardware implementation of the page table is implemented as a set of dedicated registers.
- **Page-table base register (PTBR)** points to the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction

RV College of Engineering®

Hardware Support

- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**
- The TLB is associative, high-speed memory.
- Each entry in the TLB consists of two parts:
- a key (or tag) and a value.
- If the page number is not in the TLB (known as a TLB miss), a memory reference to the page table must be made.

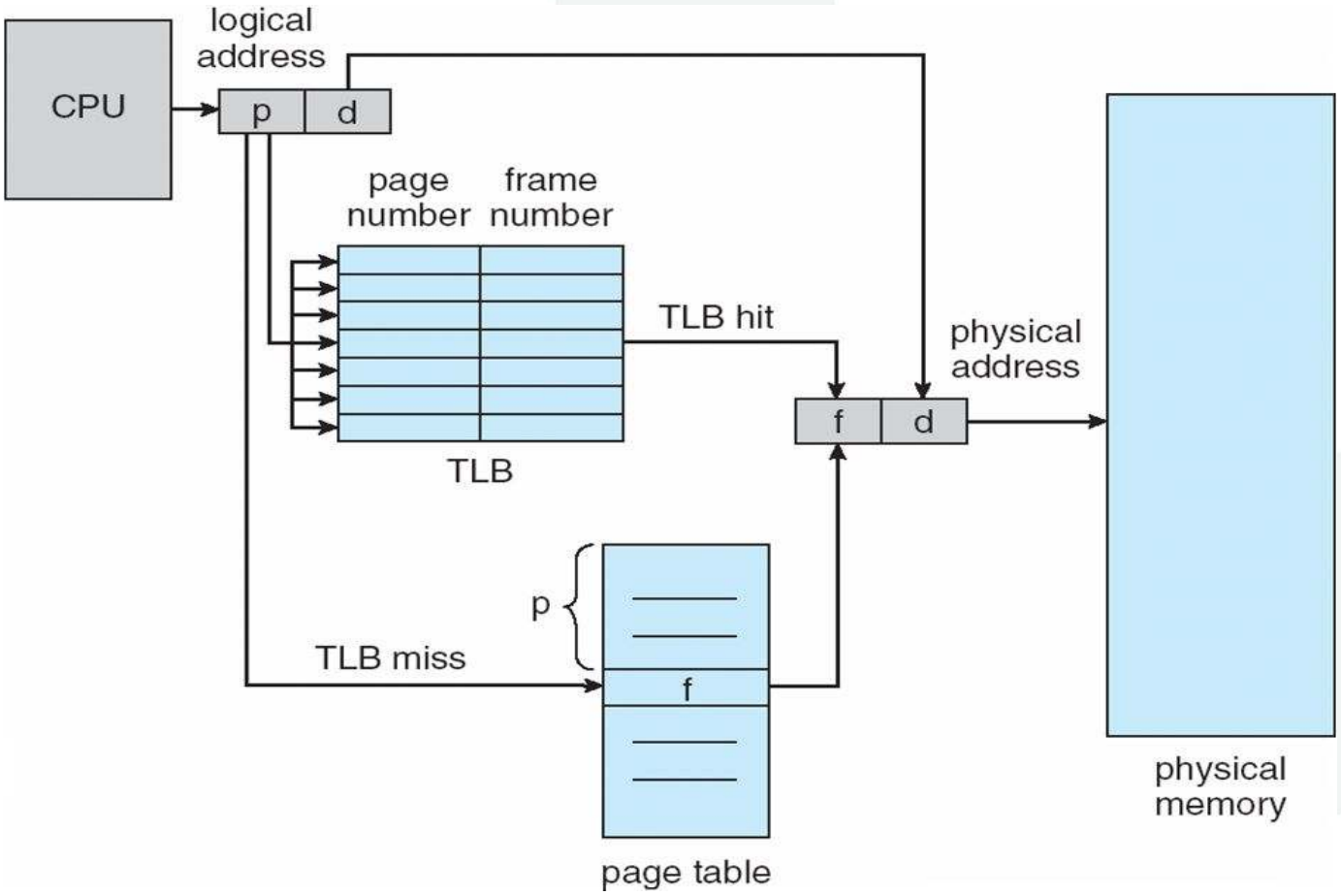
RV College of Engineering®

Hardware Support

- The new entry will be made into the TLB .
- If the memory of TLB is full then replacement takes place.
- Some portions of TLB also will be **wired down (cannot be replaced)**
- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
    - Otherwise need to flush at every context switch

RV College of Engineering®

Paging Hardware With TLB



TLB

- The percentage of times that the page number of interest is found in the TLB is called the **hit ratio.**
- In the case that the page is found in the TLB (TLB hit) the total time would be the time of search in the TLB plus the time to access memory, so
- TLB_hit_time = TLB_search_time + memory_access_time

TLB

- In the case that the page is not found in the TLB (TLB miss) the total time would be the time to search the TLB (not found anything) plus the time to access memory to get the page table and frame, plus the time to access memory to get the data, so
- TLB_miss_time=TLB_search_time+ memory_access_time + memory_access_time
- EAT = TLB_miss_time * (1- hit_ratio) + TLB_hit_time * hit_ratio

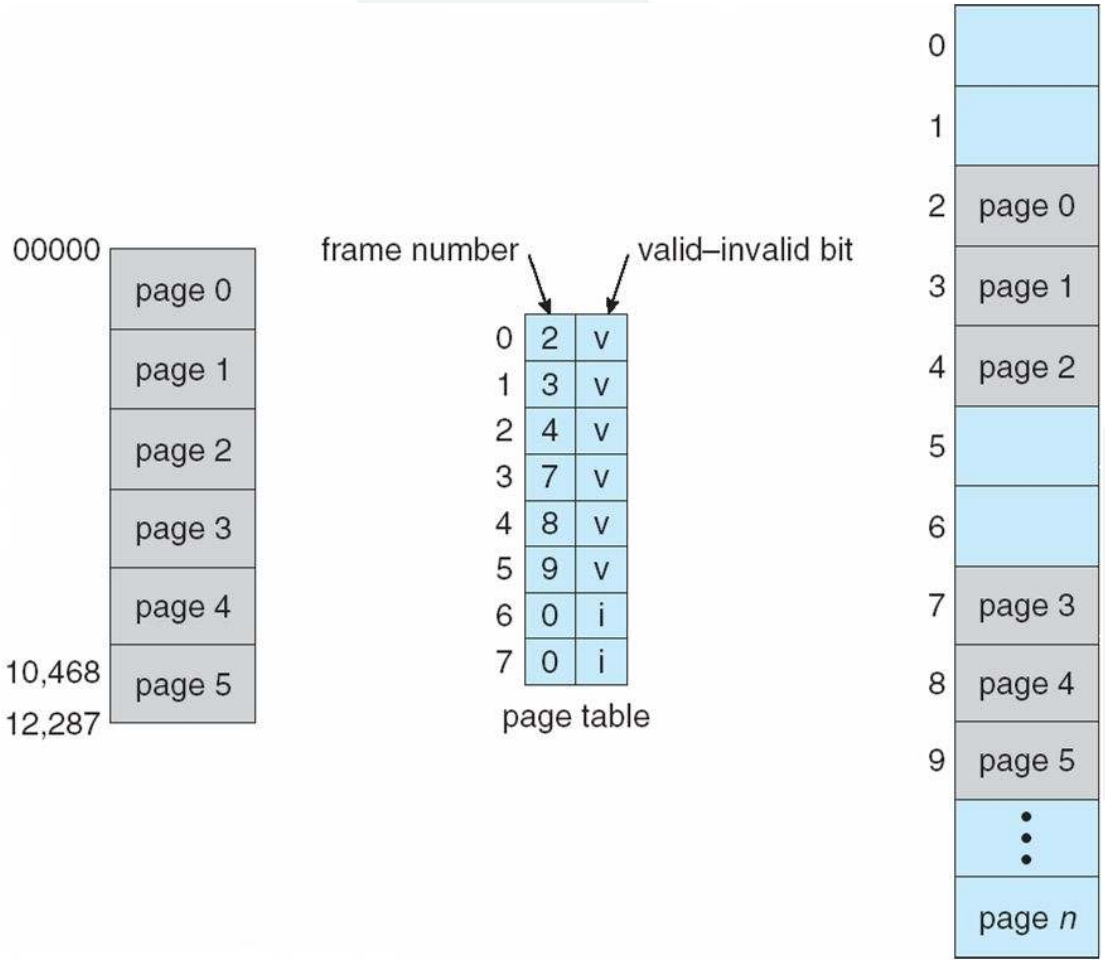
Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
- Can also add more bits to indicate page execute-only, and so on

  Valid-invalid bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space

RV College of Engineering®

Valid (v) or Invalid (i) Bit In A Page Table



Protection

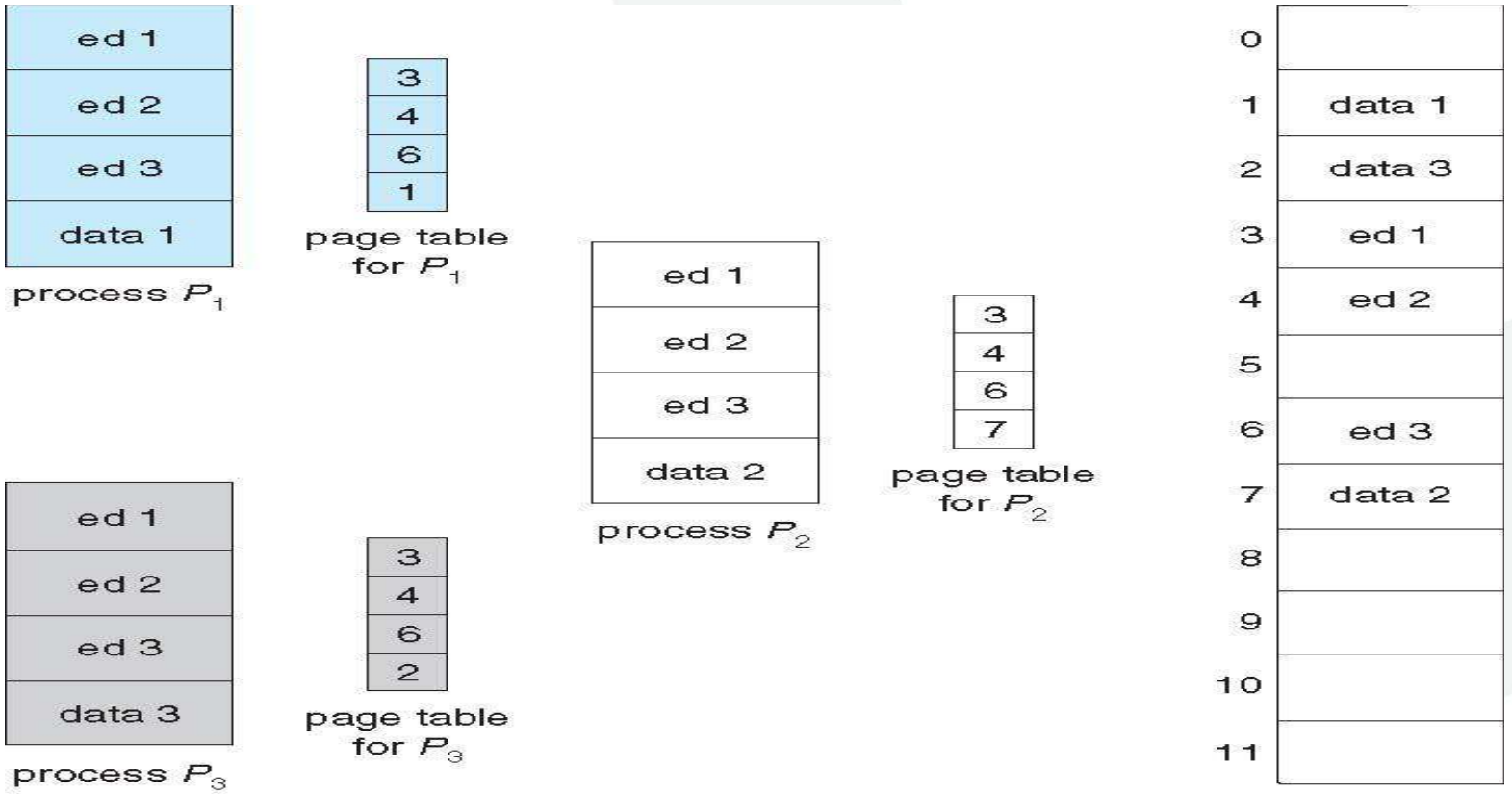
- Some systems provide hardware, in the form of a page-table length register (PTLR), to indicate the size of the page table.

Shared pages

- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
  - Similar to multiple threads sharing the same process space
  - Also useful for interprocess communication if sharing of read-write pages is allowed
- **Private code and data**
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space

RV College of Engineering®

Shared pages



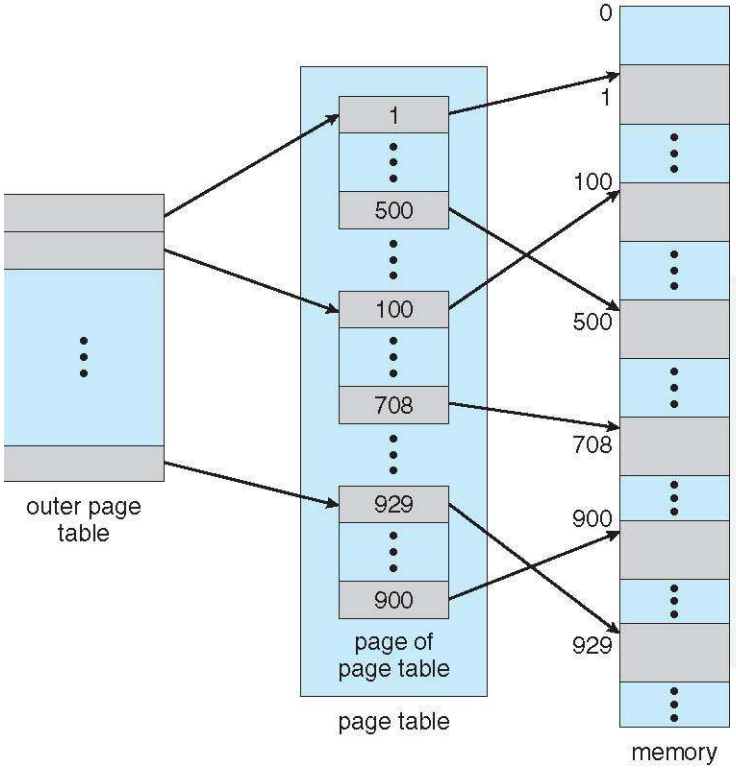
Structure of the Page Table

- hierarchical paging
- hashed page tables
- Inverted page tables.

RV College of Engineering®

hierarchical paging

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table
- Also called forward mapped page table


0
1
100
500
708
900
929
1
500
100
708
929
900
outer page table
page of page table
page table
memory

Example

- Consider that the size of main memory (physical memory) is 512 MB = 2^{29} (physical memory can be represented with 29 bits)
- This physical memory is divided into a number of frames where each frame size = 4 KB = 2^{12} (i.e. frame size can be represented with 12 bits)
- Physical memory in bits = 29
- Frame size in bits = 12
- Number of bits used to represent frame number= 29 – 12 = 17

Example

- physical memory is represented as:

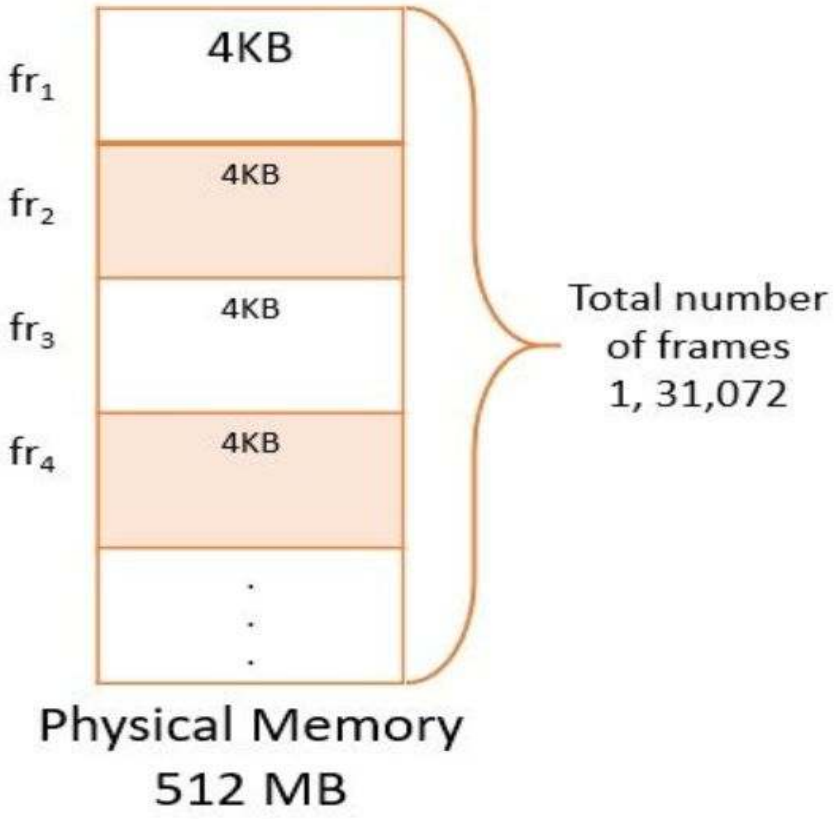
Physical Address is represented by

Frame number	Offset/Frame Size
Fr (17)	d (12)

- Total number of frames would be 2^{17} = 1, 31,072
- Now we have a physical memory of 512 MB which is divided into 131072 frames where each frame size is 4 KB.

RV College of Engineering®

Example



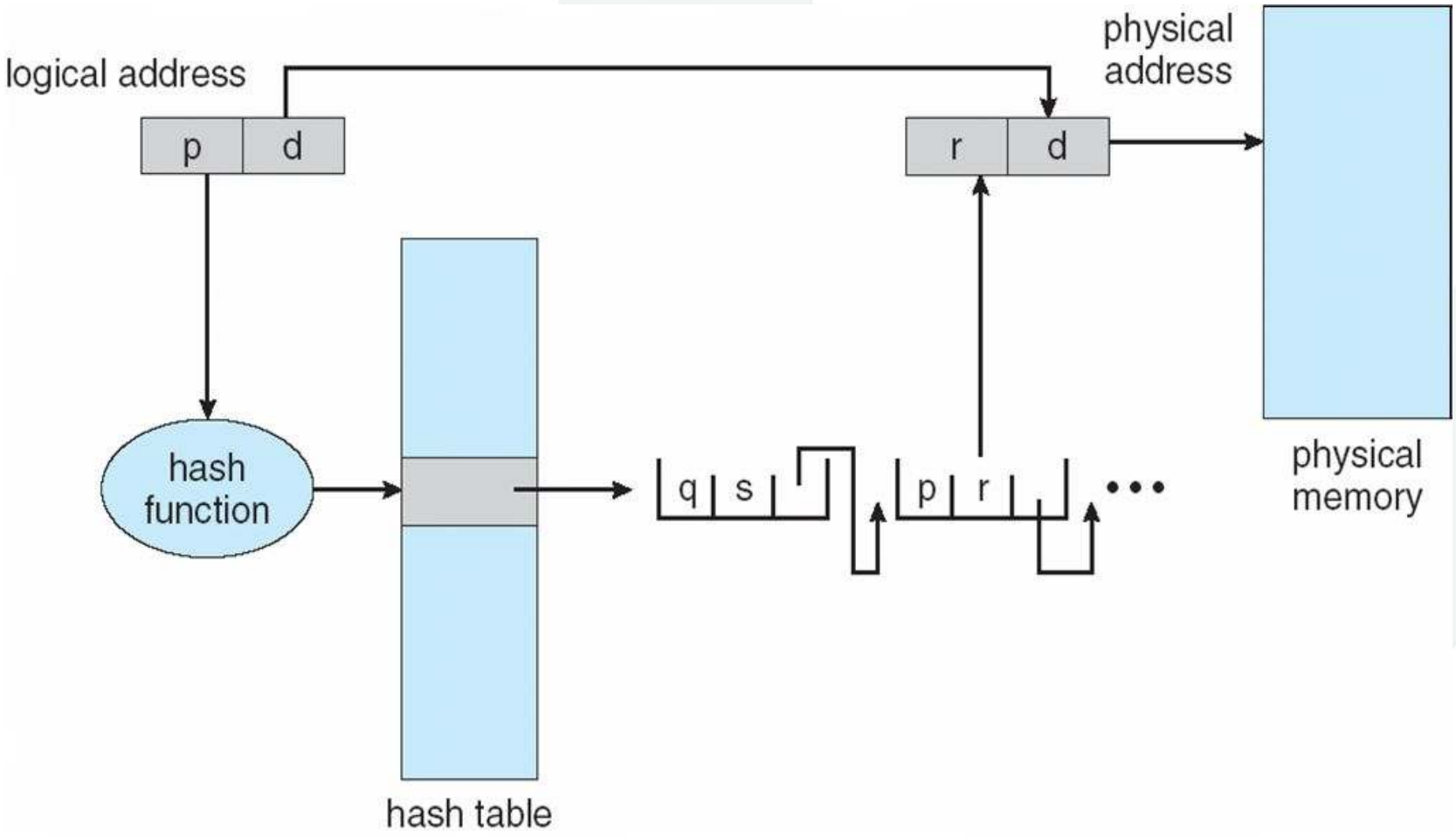
RV College of Engineering®

Hashed Page Tables

- A common approach for handling address spaces larger than 32 bits is to use a hashed page table
- The virtual page number is hashed into a page table
  - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element

Hashed Page Tables

- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, the corresponding physical frame is extracted


logical address
p
d
physical address
r
d
hash function
hash table
q
s
p
r
physical memory

Inverted Page Tables

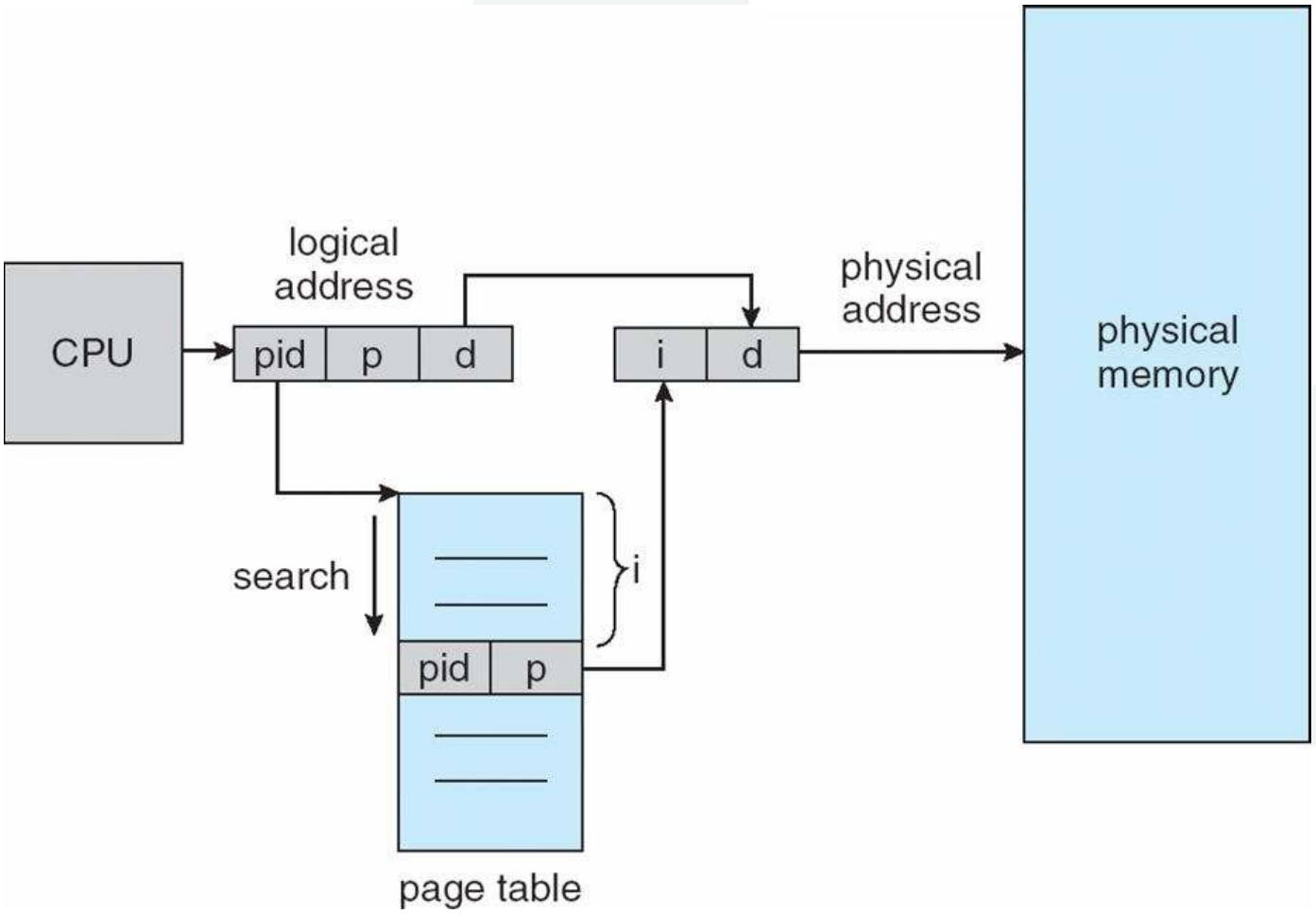
- An inverted page table has one entry for each real page (or frame) of memory.
- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page

Inverted Page Tables

- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs.
- Use hash table to limit the search to one — or at most a few — page-table entries
  - TLB can accelerate access

RV College of Engineering®

Inverted page table



Virtual Memory

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing

RV College of Engineering®

Objectives

- To describe the benefits of a virtual memory system
- To explain the concepts of demand paging, page-replacement algorithms, and allocation of page frames

Virtual Memory

- Virtual memory is a technique that allows the execution of processes that are not completely in memory.
- Code needs to be in memory to execute, but entire program rarely used
  - Error code, unusual routines, large data structures
- Entire program code not needed at same time

Virtual Memory

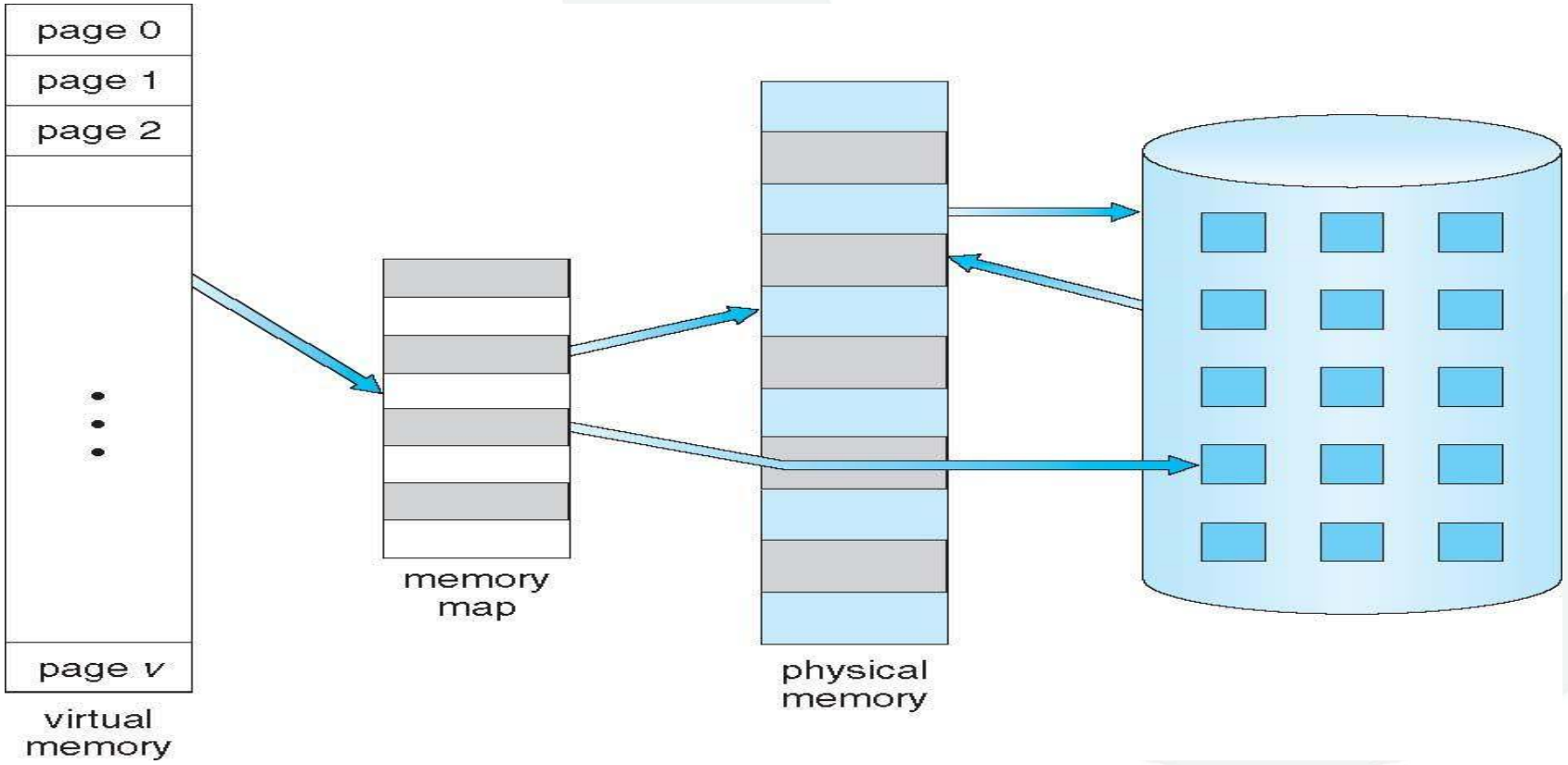
- Consider ability to execute partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running -> more programs run at the same time
    - Increased CPU utilization and throughput with no increase in response time or turnaround time
  - Less I/O needed to load or swap programs into memory -> each user program runs faster

Virtual Memory

- **Virtual memory** – separation of user logical memory from physical memory
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes
  - Allows for more efficient process creation
  - More programs running concurrently
  - Less I/O needed to load or swap processes

Virtual Memory

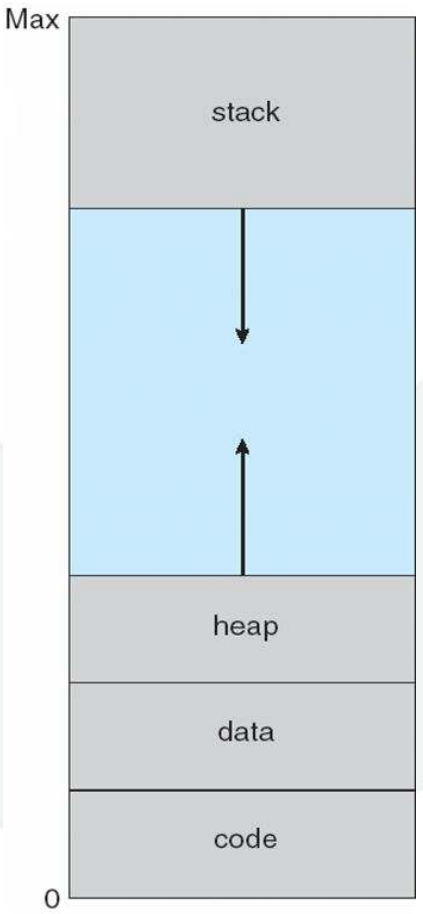
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation


page 0
page 1
page 2
page v
virtual memory
memory map
physical memory

Virtual-address Space

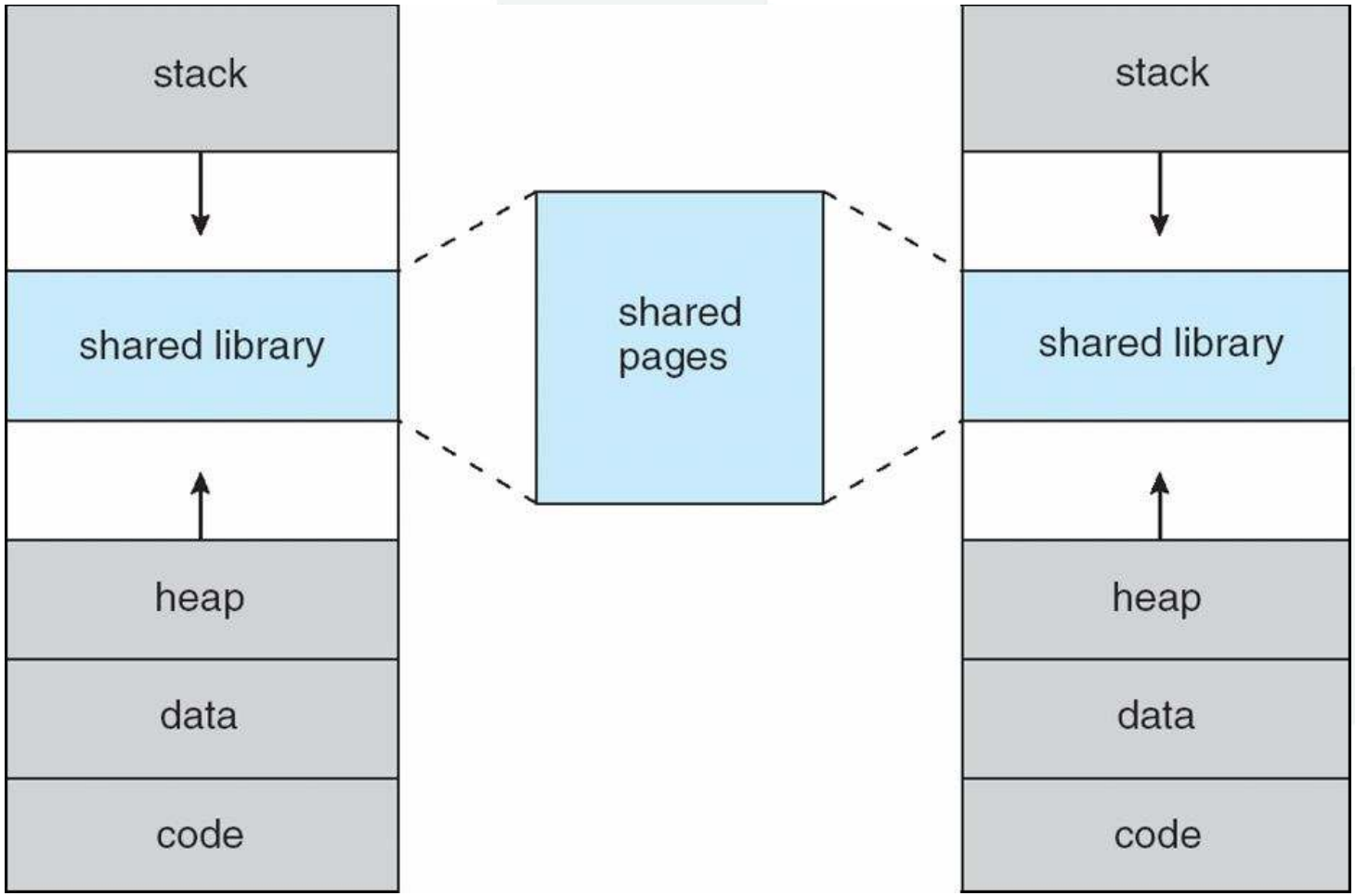
The large blank space (or hole) between the heap and the stack is part of the virtual address space but will require actual physical pages only if the heap or stack grows.

Virtual address spaces that include holes are known as sparse address spaces.



RV College of Engineering®

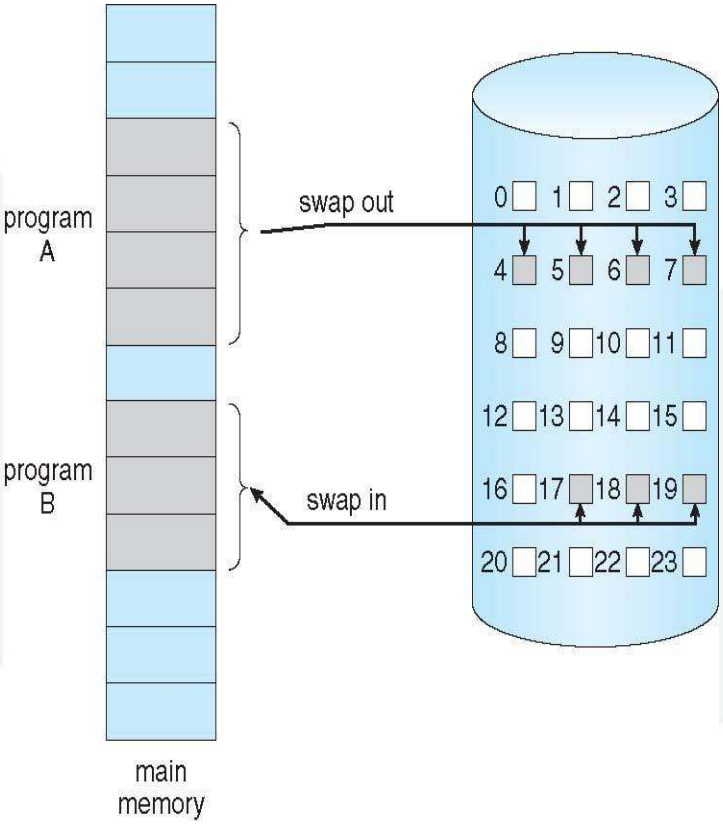
Shared Library Using Virtual Memory



RV College of Engineering®

Demand Paging

- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping (diagram on right)
- Page is needed ⇒ reference to it
  - invalid reference ⇒ abort
  - not-in-memory ⇒ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a **pager**



Basic concepts

- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- How to determine that set of pages?
  - Need new MMU functionality to implement demand paging
- If pages needed are already **memory resident**
  - No difference from non demand-paging
- If page needed and not memory resident
  - Need to detect and load the page into memory from storage
    - Without changing program behavior
    - Without programmer needing to change code

RV College of Engineering®

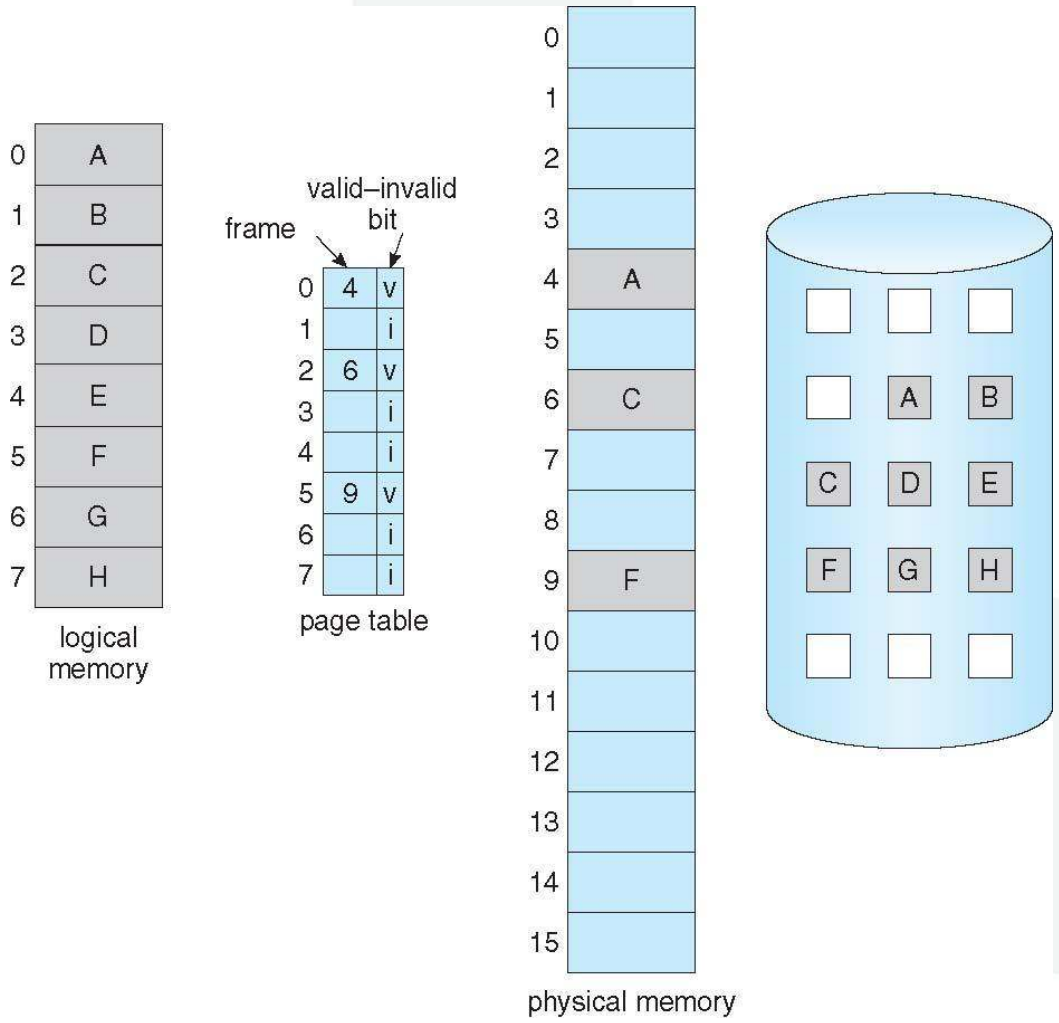
Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- During MMU address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

- Example of a page table snapshot:



Page Table When Some Pages Are Not in Main Memory

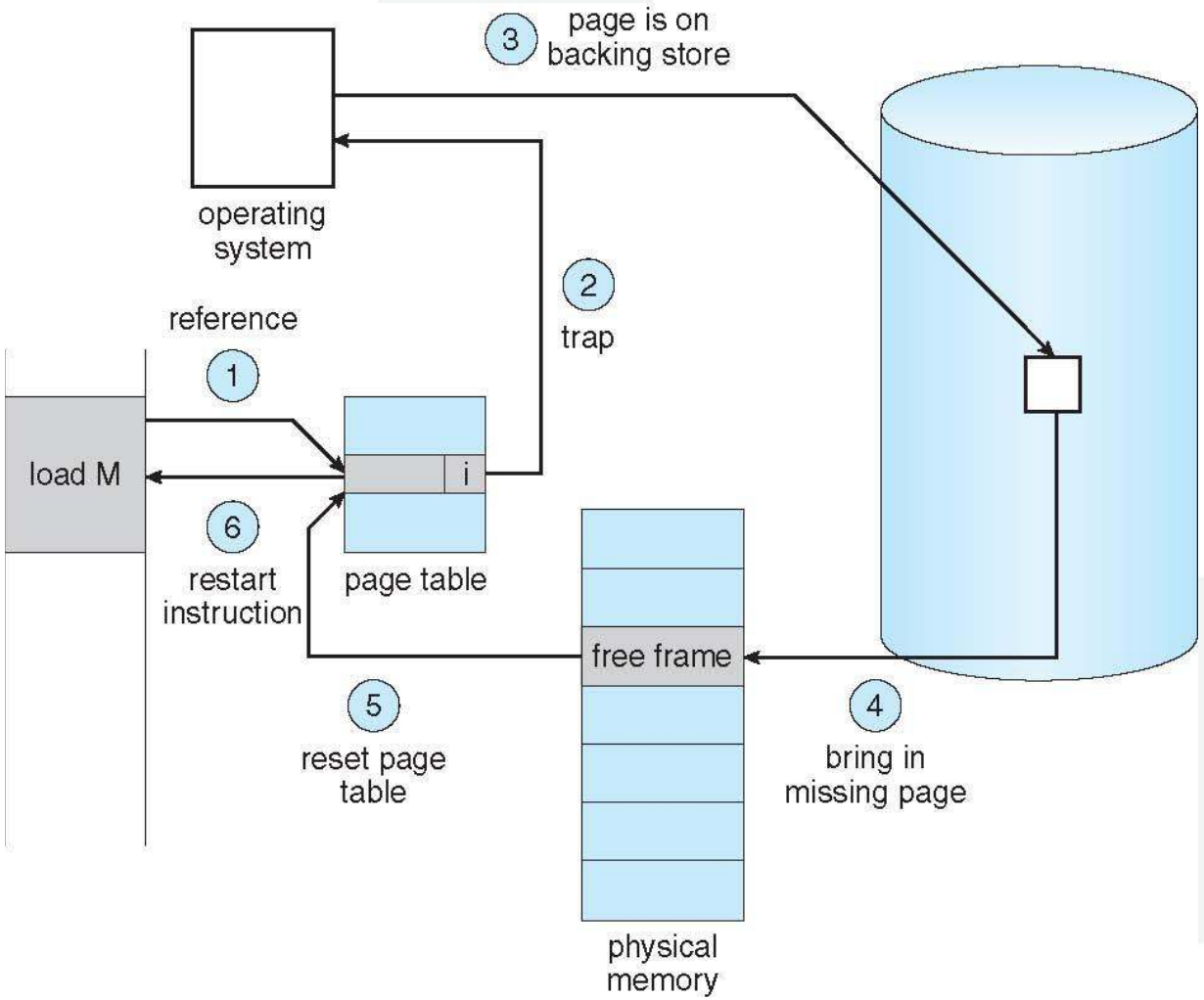


Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:
  page fault

1. Operating system looks at another table to decide:
   - Invalid reference ⇒ abort
   - Just not in memory
2. Find free frame
3. Swap page into frame via scheduled disk operation
4. Reset tables to indicate page now in memory
   Set validation bit = **v**
5. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



RV College of Engineering®

Aspects of Demand Paging

- Extreme case – start process with *no* pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
  - And for every other process pages on first access
  - **Pure demand paging**
- Actually, a given instruction could access multiple pages -> multiple page faults
  - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
  - Pain decreased because of **locality of reference**
- Hardware support needed for demand paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with **swap space**)
  - Instruction restart

Performance of Demand Paging

- Stages in Demand Paging (worse case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
    1. Wait in a queue for this device until the read request is serviced
    2. Wait for the device seek and/or latency time
    3. Begin the transfer of the page to a free frame
6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

Performance of Demand Paging (Cont.)

- Three major activities
  - Service the interrupt – careful coding means just several hundred instructions needed
  - Read the page – lots of time
  - Restart the process – again just a small amount of time
- Page Fault Rate $0 \le p \le 1$
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

  EAT = (1 – p) x memory access
  + p (page fault overhead
  + swap page out
  + swap page in)

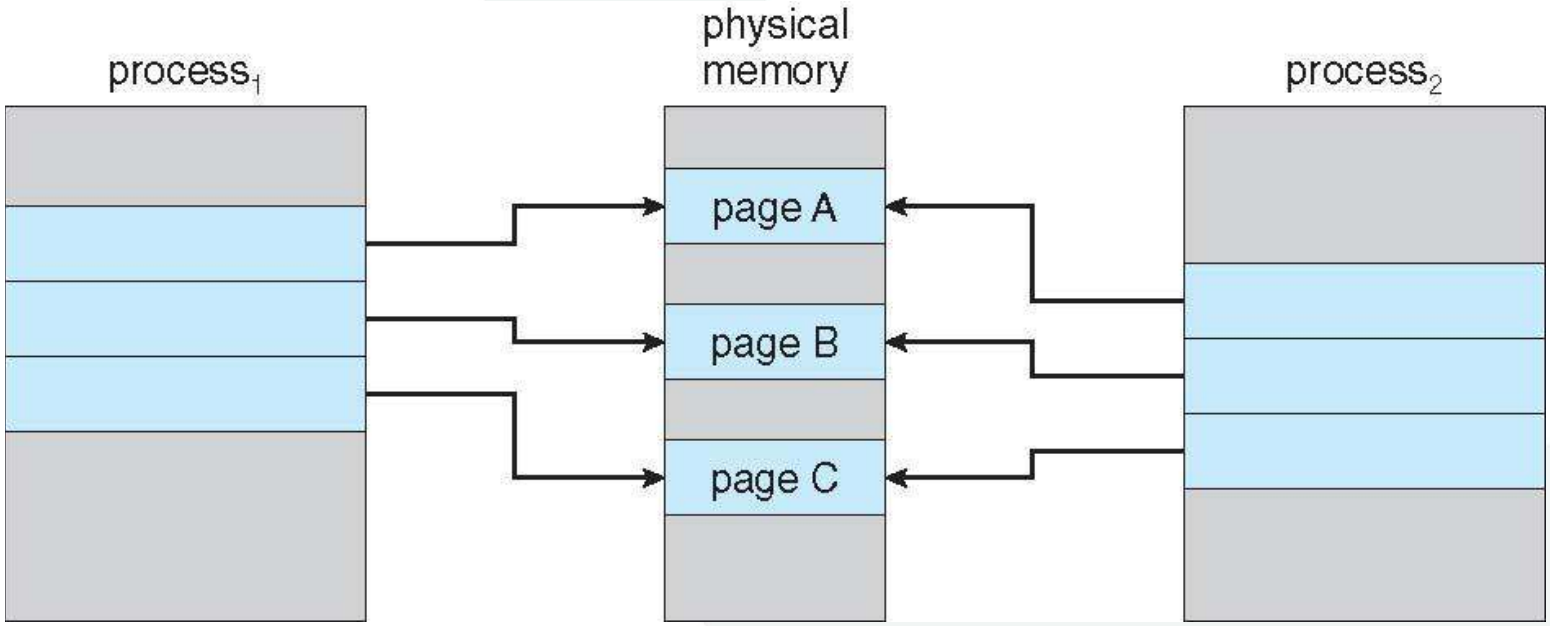
Problem

- The virtual memory system uses the demand paging for its implementation. The probability of getting page faults is 0.25, the normal memory access time is 200 nanoseconds. If it takes 2 millseconds to service a page fault, then what is effective memory access time?
- E.M.A.T. = (1 - p) * ma + p * page fault service time , where p = page fault rate
  EMAT = [0.75 * 200 + 0.25 * 2000000]ns = 500150 n.s

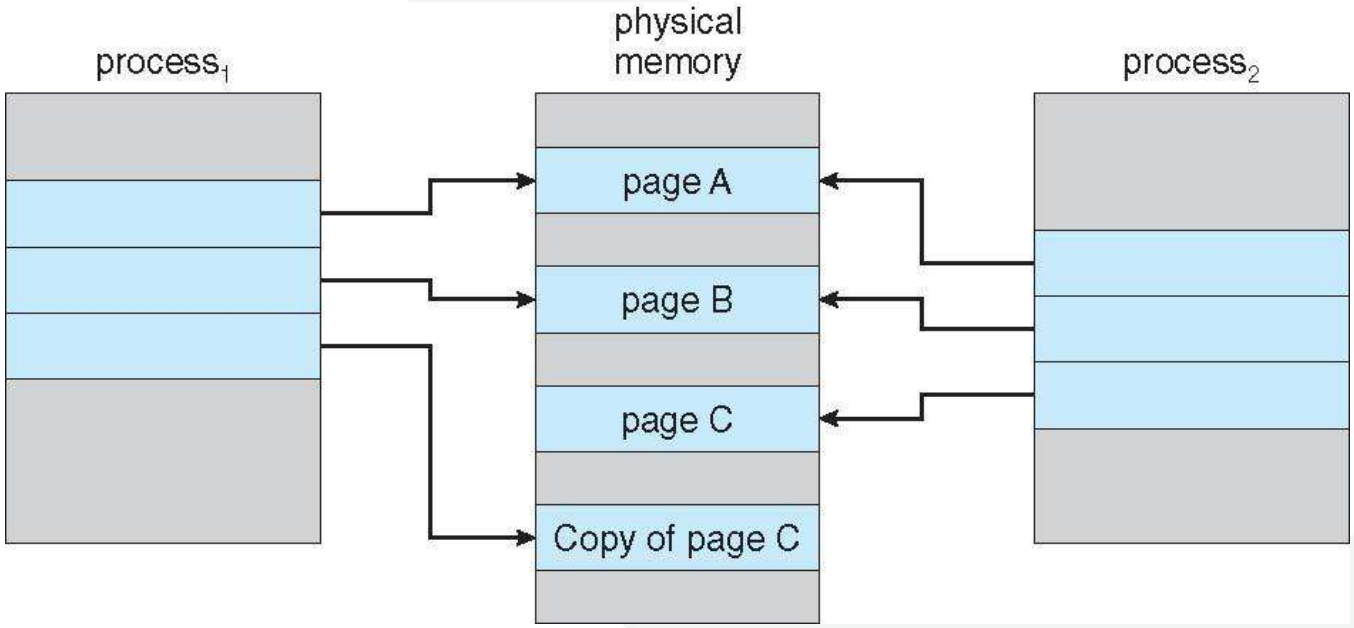
Copy-on-Write

- **Copy-on-Write** (COW) allows both parent and child processes to initially ***share*** the same pages in memory
  - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- In general, free pages are allocated from a **pool** of **zero-fill-on-demand** pages
  - Pool should always have free frames for fast demand page execution
    - Don't want to have to free a frame as well as other processing on page fault
- `vfork()` variation on `fork()` system call has parent suspend and child using copy-on-write address space of parent
  - Designed to have child call `exec()`
  - Very efficient

Before Process 1 Modifies Page C



After Process 1 Modifies Page C



What Happens if There is no Free Frame?

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
  - Algorithm – terminate? swap out? replace the page?
  - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

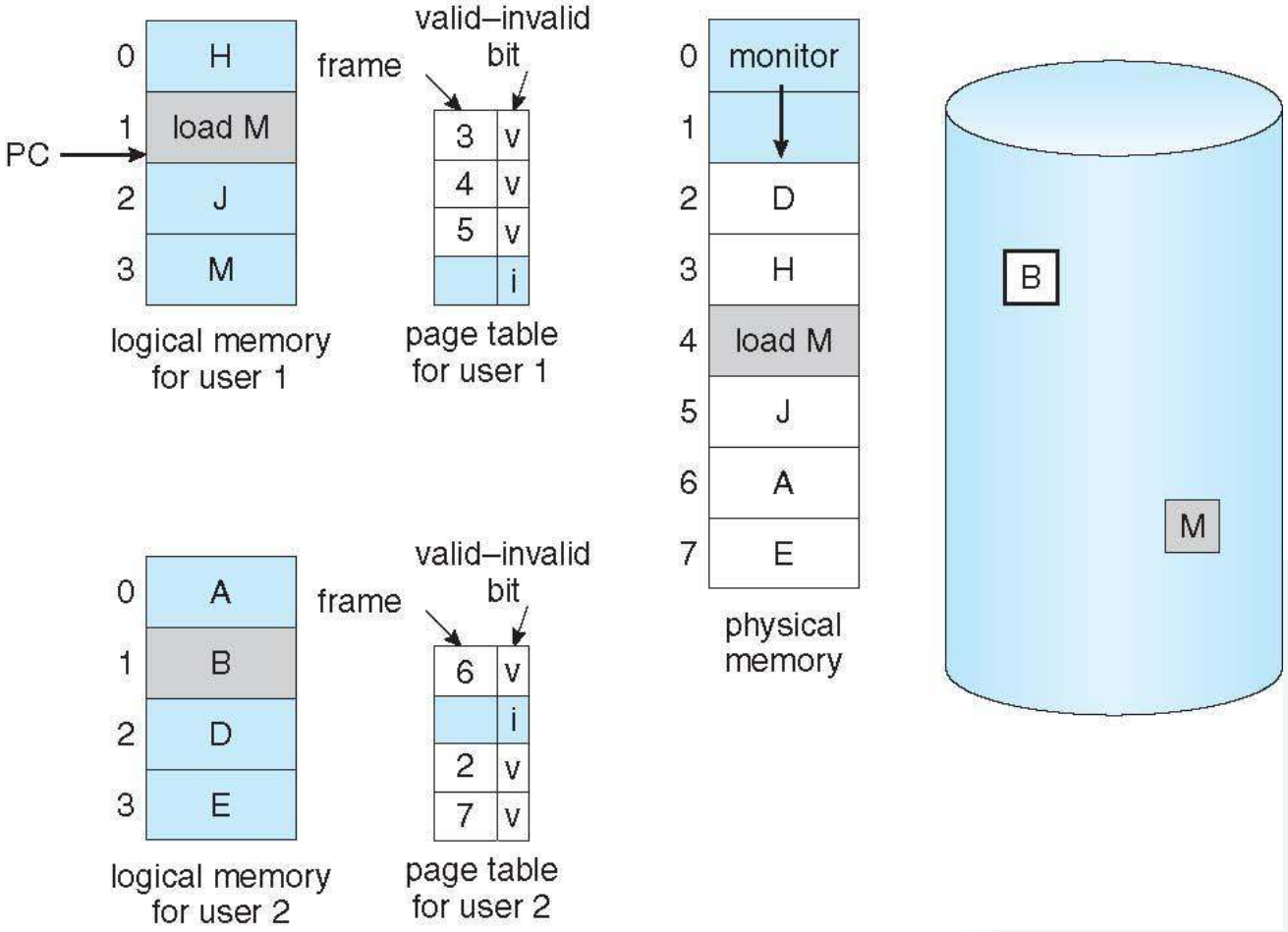
RV College of Engineering®

Page Replacement

- Prevent **over-allocation** of memory by modifying page-fault service routine to include page replacement
- Use **modify** (**dirty**) **bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory

RV College of Engineering®

Need For Page Replacement



Basic Page Replacement

1. Find the location of the desired page on disk

2. Find a free frame:
   - If there is a free frame, use it
   - If there is no free frame, use a page replacement algorithm to select a **victim frame**
   - Write victim frame to disk if dirty

3. Bring the desired page into the (newly) free frame; update the page and frame tables

4. Continue the process by restarting the instruction that caused the trap

Note now potentially 2 page transfers for page fault – increasing EAT

RV College of Engineering®

Page Replacement

